

Universitat de Girona
Escola Politècnica Superior

Grau en Enginyeria Informàtica

PROJECTE FINAL DE GRAU

**LSigner, una plataforma integral de gestió i
verificació de signatura digital**

Autor/a:
Sr. Ramon López Cros

Tutor/a:
Dr. Antoni Bardera Reig

MEMÒRIA

Convocatòria:
20 de juliol del 2026

Departament:
Departament d'Informàtica, Matemàtica Aplicada i Estadística

© 2026 Sr. Ramon López Cros. Tots els drets reservats.

Aquesta obra és propietat exclusiva de l'autor. Es lliura únicament amb finalitat acadèmica, per a la seva avaluació com a Projecte Final de Grau. Se'n prohibeix la reproducció, la distribució, la comunicació pública i la transformació, totals o parcials, per qualsevol mitjà o suport, sense l'autorització expressa i per escrit de l'autor, llevat de la seva conservació i exposició per part de la Universitat de Girona a la biblioteca universitària, amb finalitats de custòdia i consulta acadèmica.

Projecte: Projecte Final de Grau
Document: Memòria
Títol: LSigner, una plataforma integral de gestió i verificació de signatura digital
Autor: Sr. Ramon López Cros
Data: 20 de juliol del 2026

Estudi:
Grau en Enginyeria Informàtica
Universitat de Girona

Tutor/a:
Dr. Antoni Bardera Reig
Universitat de Girona
Email: anton.bardera@udg.edu
Web: [Link](#)

Agraïments

Aquestes línies són per a totes les persones que, d'una manera o altra, han fet possible aquest projecte o m'han ajudat a continuar endavant.

En primer lloc, al meu tutor, el Dr. Antoni Bardera, per la seva flexibilitat sempre que he necessitat res, per escoltar les meves explicacions sense fi de tot allò que anava implementant, i per acceptar la proposta d'aquest projecte a només pocs dies de tancar-se'n el termini de presentació. També li vull agrair la seva feina a l'assignatura de Multimèdia i Interfícies d'Usuari, on em va ensenyar un primer tast de desenvolupament web i on, en certa manera, va ser el meu precursor en aquest camí.

A la coordinadora del grau, la Dra. Marta Fort, pel seu suport amb els meus dubtes sobre com orientar la defensa del projecte i sobre com fer arribar al tribunal tot el valor de la feina feta, en un moment en què això, amb la intel·ligència artificial pel mig, no és gens fàcil.

En especial, als meus professors de batxillerat, en Jordi Baró Romà, de química, i en Josep Ribas, de física, els meus dos grans referents d'admiració. Més enllà de les seves assignatures, em van ensenyar a aprendre. En ells dos vaig veure una passió pel que ensenyaven, una passió que de seguida se'm va enganxar i que no m'ha abandonat mai: encara és l'espurna que m'acompanya avui en cada projecte que emprenc. Sempre els porto al cor.

«Quien ignora los principios de la Geometría no será capaz de distinguir un redondel de una circunferencia, del mismo modo que quien ignora la Teoría de la Literatura no será capaz de distinguir una obra literaria de un código de barras, pues supondrá derridianamente que todo es texto.»

Jesús G. Maestro

«La creatividad es la inteligencia divirtiéndose.»

Joaquín Domínguez Portela

Índex

Agraïments	iii
Índex de figures	viii
Índex de taules	ix
Llistat d'acrònims	x
1 Introducció, motivacions, propòsit i objectius del projecte	1
1.1 Introducció	1
1.2 Motivacions	2
1.3 Propòsit	2
1.4 Objectius	2
1.5 Estructura del document	3
2 Estudi de viabilitat	4
2.1 Estudi de mercat	4
2.2 Viabilitat tècnica	5
2.3 Viabilitat econòmica	6
2.4 Viabilitat legal	7
2.5 Viabilitat ètica i ús d'IA	7
2.6 Anàlisi de riscos	8
2.7 Conclusió	9
3 Metodologia	10
3.1 Metodologia de treball	10
3.1.1 Definició adaptativa de l'abast	10
3.2 Flux de treball i control de versions	11
3.3 Cicle de desenvolupament	11
3.4 Ús d'eines d'intel·ligència artificial	12
4 Planificació	14
4.1 Estratègia de planificació	14
4.2 Paquets de treball	15
4.2.1 PT1: <i>Backend</i>	15
4.2.2 PT2: Infraestructura i DevOps	15
4.2.3 PT3: <i>Frontend</i>	16
4.2.4 PT4: Iteració final i memòria	17
4.3 Cronograma	18
5 Marc de treball i conceptes previs	19
5.1 Marc de treball	19

5.1.1	Persones involucrades	19
5.1.2	Coneixements i experiència previs	19
5.2	Conceptes previs	20
5.2.1	La signatura electrònica i el marc legal	20
5.2.2	Fonaments criptogràfics	21
5.2.3	Arquitectura d'aplicacions web	22
5.2.4	Contenidors i desplegament	23
5.2.5	Integració i desplegament continu (CI/CD)	24
5.2.6	Proves de programari	25
6	Requisits del sistema	26
6.1	Requisits funcionals	26
6.2	Requisits no funcionals	28
7	Estudis i decisions	30
7.1	Maquinari	30
7.2	Pila del <i>backend</i>	30
7.2.1	Node.js i NestJS	30
7.2.2	TypeORM i la gestió de transaccions	31
7.2.3	PostgreSQL	31
7.2.4	Criptografia: Ed25519	31
7.3	Pila del <i>frontend</i>	32
7.3.1	Next.js i React	32
7.3.2	MUI i Tailwind	32
7.3.3	Client d'API propi	33
7.4	Infraestructura i desplegament	33
7.4.1	VPS enfront de plataformes gestionades	33
7.4.2	Dokploy	33
7.4.3	Emmagatzematge: Garage	33
7.5	Eines de desenvolupament i qualitat	34
8	Anàlisi i disseny del sistema	35
8.1	Anàlisi	35
8.1.1	Tipus d'usuari i rols	35
8.1.2	Anàlisi dels requisits	35
8.1.3	Casos d'ús	36
8.1.4	Model conceptual de dades	37
8.2	Disseny	38
8.2.1	Disseny de la base de dades	38
8.2.2	Disseny de les interfícies d'usuari	40
8.2.3	Patrons de disseny aplicats	41
9	Implementació i proves	42
9.1	Arquitectura general del sistema	42
9.2	Desenvolupament dels mòduls principals	43
9.2.1	El nucli de signatura: payload canònic i Ed25519	43
9.2.2	Verificació d'identitat (OTP) i el seu vincle amb la firma	44
9.2.3	Gestió de sessió al <i>frontend</i> : refresc concurrent	45
9.2.4	Docker multietapa	46
9.3	Integració	46
9.4	Proves i validació	47

9.4.1	Estratègia de <i>testing</i>	47
9.4.2	Cobertura i exemples representatius	48
9.5	Codi font i vídeo	48
10	Implantació i resultats	49
10.1	Procés d'implantació	49
10.1.1	Primers passos: el <i>backend</i>	49
10.1.2	Aixecament de la infraestructura de producció	49
10.1.3	Desenvolupament i desplegament del <i>frontend</i>	50
10.1.4	El proveïdor de correu electrònic	50
10.1.5	Tancament: proves reals i redacció de la memòria	50
10.2	Compliment legal i normatiu	50
10.3	Assoliment dels objectius	51
10.4	Assoliment dels requisits	52
11	Conclusions	53
11.1	Valoració global	53
11.2	Principals resultats	53
11.3	Desviacions de la planificació	54
11.4	Aportacions personals i aprenentatge	54
11.5	Reflexió final	55
12	Treball futur	56
12.1	Treball pendent	56
12.2	Noves línies d'evolució	57
12.3	Cap a un producte real	57
A	Fitxes de casos d'ús	59
B	Disseny detallat de la base de dades	62
C	Detalls addicionals d'implementació	66
C.1	Sessions públiques per a destinataris externs	66
C.2	El <i>symlink</i> de Chromium a Alpine per a Playwright	67
	Bibliografia	69

Índex de figures

3.1	Cicle de desenvolupament i desplegament continu	11
3.2	Comprovacions del <i>pipeline</i> d'integració contínua	12
4.1	Cronograma del projecte (diagrama de Gantt)	18
8.1	Diagrama de casos d'ús del sistema	37
8.2	Model conceptual de dades del sistema	38
9.1	Arquitectura general del sistema	42
9.2	Seqüència de firma d'un document	46
9.3	Topologia de desplegament	47

Índex de taules

2.1	Pressupost estimat del projecte.	6
4.1	Paquet PT1 (<i>backend</i>)	15
4.2	Paquet PT2 (infraestructura i DevOps)	16
4.3	Paquet PT3 (<i>frontend</i>)	16
4.4	Paquet PT4 (iteració final i memòria)	17
6.1	Requisits funcionals del sistema.	26
6.2	Requisits no funcionals del sistema.	28
8.1	Disseny de la taula <code>documents</code>	38
8.2	Disseny de la taula <code>document_recipients</code>	39
8.3	Disseny de la taula <code>document_signed_artifacts</code>	39
8.4	Disseny de la taula <code>document_signing_events</code>	40
9.1	Nombre de proves i cobertura de codi	48
B.1	Disseny de la taula <code>users</code>	62
B.2	Disseny de la taula <code>contacts</code>	62
B.3	Disseny de la taula <code>refresh_tokens</code>	63
B.4	Disseny de la taula <code>document_locks</code>	63
B.5	Disseny de la taula <code>document_lock_resolutions</code>	63
B.6	Disseny de la taula <code>otp_challenges</code>	64
B.7	Disseny de la taula <code>public_link_sessions</code>	65

Llistat d'acrònims

ACID	Atomicitat, consistència, aïllament i durabilitat (<i>Atomicity, Consistency, Isolation, Durability</i>)
AES	Signatura electrònica avançada (<i>Advanced Electronic Signature</i>)
API	<i>Application Programming Interface</i>
B2B	<i>Business to Business</i>
CI/CD	Integració contínua i desplegament continu (<i>Continuous Integration / Continuous Deployment</i>)
CRM	<i>Customer Relationship Management</i>
CRUD	Crear, llegir, actualitzar i esborrar (<i>Create, Read, Update, Delete</i>)
E2E	Proves d'extrem a extrem (<i>End-to-End</i>)
ECDSA	<i>Elliptic Curve Digital Signature Algorithm</i>
eIDAS	<i>Electronic Identification, Authentication and Trust Services</i> (Reglament UE 910/2014)
ENS	Esquema Nacional de Seguretat
ERP	<i>Enterprise Resource Planning</i>
GEINF	Grau en Enginyeria Informàtica
IA	Intel·ligència artificial
ISO	<i>International Organization for Standardization</i>
JWT	<i>JSON Web Token</i>
Kanban	Mètode de gestió visual del flux de treball
KVM	<i>Kernel-based Virtual Machine</i>
LSSICE	Llei de serveis de la societat de la informació i de comerç electrònic
MVC	<i>Model-View-Controller</i>
ORM	<i>Object-Relational Mapping</i>
OTP	Contrasenya d'un sol ús (<i>One-Time Password</i>)
PaaS	Plataforma com a servei (<i>Platform as a Service</i>)
PR	<i>Pull Request</i>
QES	Signatura electrònica qualificada (<i>Qualified Electronic Signature</i>)
RBAC	Control d'accés basat en rols (<i>Role-Based Access Control</i>)
RGPD	Reglament general de protecció de dades
RSA	Esquema criptogràfic de clau pública (<i>Rivest-Shamir-Adleman</i>)
S3	<i>Amazon Simple Storage Service</i>
SaaS	<i>Software as a Service</i>
SAST	<i>Static Application Security Testing</i>
Scrum	Marc de treball àgil per a la gestió de projectes
SES	Signatura electrònica simple (<i>Simple Electronic Signature</i>)
SMS	<i>Short Message Service</i>

SOLID	Cinc principis de disseny orientat a objectes (<i>Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion</i>)
SPA	Aplicació de pàgina única (<i>Single-Page Application</i>)
SSR	Renderització al servidor (<i>Server-Side Rendering</i>)
TLS	<i>Transport Layer Security</i>
UAT	<i>User Acceptance Testing</i>
VPS	Servidor privat virtual (<i>Virtual Private Server</i>)

Capítol 1

Introducció, motivacions, propòsit i objectius del projecte

1.1 Introducció

L'era digital està portant a l'entorn electrònic diversos procediments que tradicionalment han estat lligats al paper: contractes, control d'assistència, factures, consentiments mèdics i tota mena de documents d'àmbit jurídic o administratiu. Avui en dia, molts d'aquests documents ja es generen, s'intercanvien i se signen de manera digital, gràcies a la signatura electrònica.

Un document signat electrònicament porta associat un conjunt de dades que permet identificar el signant i garantir la seva conformitat amb el contingut signat. A escala europea, el seu ús està regulat pel Reglament (UE) 910/2014, conegut com a eIDAS [1], que estableix un marc legal comú i distingeix diferents nivells de garantia: signatura simple, avançada i qualificada, segons la robustesa dels mecanismes emprats. Tècnicament, les solucions més sòlides es recolzen en la criptografia asimètrica i les funcions de *hash*, uns fonaments que fan detectable qualsevol alteració del document signat i que es descriuen al Capítol 5. La seguretat de la signatura electrònica es basa en la dificultat de vulnerar totes les barreres de seguretat de l'usuari que signa originalment: identificació, contrasenya, multifactor (si n'hi ha) i, en els casos de signatura avançada o qualificada, possessió de la clau criptogràfica de l'usuari.

El problema real que aquesta solució resol és el de la confiança en un entorn on els documents es poden copiar, modificar i transmetre sense deixar rastre visible. Una signatura electrònica, ben implementada, aporta diverses garanties fonamentals (integritat, autenticitat, traçabilitat i, en els nivells més alts, no-repudi), que es defineixen en detall al Capítol 5. Aquestes garanties són crítiques en sectors on un document mal signat o manipulat té conseqüències legals, econòmiques o de seguretat directes, com ara l'àmbit legal, el sanitari, l'educatiu o l'empresarial.

Malgrat l'evolució d'aquesta tecnologia, el mercat actual de solucions presenta un gran buit: sovint prioritza una de les dues cares del problema (la seguretat o l'experiència d'usuari) a costa de l'altra, o bé s'assenta sobre bases tècniques difícils d'entendre, mantenir i integrar. Aquest projecte parteix d'aquesta observació per plantejar una alternativa desenvolupada íntegrament des de zero, cobrint totes les capes del sistema: *backend*, *frontend* i infraestructura de desplegament, tot posant el focus en la qualitat d'enginyeria de tot el procés, no només en el resultat funcional.

1.2 Motivacions

La motivació d'aquest projecte s'origina en la meua experiència professional en el sector de la salut, concretament en l'àmbit *healthtech* [2], on he treballat amb diversos proveïdors de signatura electrònica. Aquesta experiència em va permetre identificar una mancança recurrent: moltes d'aquestes solucions sacrifiquen la usabilitat en favor de la seguretat o al revés, i acaben sent eines poc amigables o bé insuficientment robustes. D'altres són tan funcionals com segures, però estan construïdes sobre una arquitectura desordenada, difícil d'entendre, de mantenir i d'integrar.

Com a antecedent al projecte, vaig desenvolupar, en l'entorn professional, una prova de concepte de consentiment informat electrònic basada en tecnologia *blockchain* (mitjançant el *framework* Hyperledger Besu [3], amb contractes intel·ligents [4] i protocols de signatura). Després d'analitzar la complexitat i els terminis d'implementació que una solució d'aquest tipus exigia per a un projecte al qual només es destinava un desenvolupador, l'empresa va decidir retirar el projecte i emprar una solució ja existent de la mà de Telefónica Tech. Tot i que la iniciativa es va cancel·lar, del projecte em vaig emportar un coneixement tècnic més profund sobre els fonaments criptogràfics, els protocols de signatura electrònica i les arquitectures *blockchain*.

LSigner neix, per tant, fruit d'aquesta necessitat i, alhora, com a repte personal: portar una idea pròpia, des de zero, fins a un producte amb una qualitat d'enginyeria equiparable a la d'un sistema real en producció. LSigner no aspira a ser un simple prototip funcional d'una idea, sinó un producte professional plenament preparat per a un entorn de producció.

1.3 Propòsit

El propòsit d'aquest projecte és desenvolupar una plataforma de signatura electrònica completa (*backend*, *frontend* i infraestructura) que compleixi els estàndards de l'enginyeria del programari. Tal com es va establir en la proposta inicial, «no es busca la millor funcionalitat, sinó la qualitat tècnica»: sense renunciar a una funcionalitat completa i realista, l'èmfasi es posa en el rigor del disseny mitjançant l'aplicació de principis SOLID, arquitectura neta [5] i codi robust. La meta és un sistema professional i preparat per a producció, que permeti gestionar la signatura de documents garantint la integritat, l'autenticitat i la traçabilitat del procés, amb evidències de cada acció orientades al no-repudi.

1.4 Objectius

L'abast d'aquest projecte no es limita purament al producte final, sinó que posa molt d'èmfasi en el procés d'aprenentatge d'enginyeria, la metodologia, els fluxos de treball i la infraestructura, de manera que podem definir els objectius en dos grups.

Els objectius del procés:

- Aplicar pràctiques de codi net [6]: patrons de disseny i bones pràctiques de desenvolupament.
- Implementar metodologies de treball professionals: gestió àgil adaptada [7], seguretat i DevOps [8].

- Dissenyar i configurar una cadena d'integració contínua i desplegament continu [9]: GitHub Actions [10], contenidorització amb Docker [11] i desplegament en VPS.
- Experimentar el cicle de vida complet d'un producte propi, des del desenvolupament fins a la posada en producció, i consolidar l'aprenentatge de les pràctiques professionals que això comporta.
- Dur a terme amb èxit les integracions amb els serveis externs i d'infraestructura que la plataforma necessita: correu electrònic, domini, servidor virtual (VPS) i plataforma de desplegament (Dokploy [12]).

Els objectius del producte:

- Assolir una plataforma de signatura electrònica amb qualitat de producció: una arquitectura segura, modular i mantenible.
- Aconseguir que la signatura de documents assoleixi un nivell de seguretat robust, sustentat en evidències verificables de manera independent i orientades al no-repudi, tot complint la normativa vigent de protecció de dades i seguretat (RGPD [13] i principis de l'ENS [14]).
- Desplegar i mantenir el producte final en un entorn de producció real i estable, amb una infraestructura que en garanteixi el funcionament continuat i la fiabilitat.

Aquest conjunt d'objectius forma una mena de contracte previ al desenvolupament i busca servir de guia per a tot el procés. Així doncs, els capítols següents giren al voltant d'aquests objectius.

1.5 Estructura del document

Aquesta memòria estructura el seu contingut en dotze capítols que es poden agrupar en quatre blocs segons el tema.

El primer bloc estableix el marc inicial del projecte: aquest capítol introductori en recull les motivacions, el propòsit i els objectius; l'estudi de viabilitat n'analitza la factibilitat; i els capítols de metodologia i planificació detallen, respectivament, la manera de treballar i la temporització prevista.

El segon bloc en fixa els fonaments i l'especificació, amb el marc de treball i els conceptes previs, els requisits del sistema i els estudis i decisions que justifiquen les tecnologies i solucions que s'han emprat.

El tercer bloc constitueix el nucli d'enginyeria del treball: l'anàlisi i el disseny del sistema, la seva implementació i proves, així com la implantació i els resultats obtinguts.

Finalment, el darrer bloc en recull les conclusions i el treball futur, que planteja les línies d'evolució de la plataforma així com propostes obertes.

Capítol 2

Estudi de viabilitat

Abans de l'inici del projecte cal estudiar-ne la viabilitat i assegurar que es pot implementar sense cap problema. Per a dur a terme l'anàlisi, s'actuarà en el context d'una *startup* tecnològica que veu potencial en aquest sector del mercat i vol tenir control total del procés de desenvolupament i desplegament del seu producte. El fet que aquest projecte pugui distingir la part purament de codi i desenvolupament de la part de gestió d'infraestructura i desplegament fa que l'anàlisi de viabilitat s'hagi d'abordar des d'una doble perspectiva: d'una banda, la del programari que s'ha de construir; i de l'altra, la de la infraestructura que l'ha de sostenir i posar en producció. Ambdues perspectives comparteixen objectius, però impliquen costos, riscos i decisions tècniques diferents, de manera que cal valorar-les primer per separat i, després, de manera conjunta. Així doncs, el projecte serà realment viable si ho són les dues perspectives alhora. Els apartats següents n'analitzen el mercat, la viabilitat tècnica, econòmica, legal i ètica, i tanquen amb una anàlisi de riscos que enumera les amenaces principals i les mesures previstes per mitigar-les.

2.1 Estudi de mercat

La signatura electrònica és un mercat consolidat i madur que ja té diverses solucions funcionant arreu del món. És un sector prou competitiu on es troben tant grans plataformes internacionals com proveïdors especialitzats de l'àmbit europeu. A tall d'exemple, podem destacar-ne tres molt emprats a l'Estat espanyol:

- **VIDsigner** (Validated ID) [15]: proveïdor europeu que ofereix diverses modalitats de signatura (biomètrica sobre una tauleta física, remota amb autenticació multifactor i amb certificat qualificat). VIDsigner és un prestador qualificat de serveis de confiança segons l'eIDAS, cosa que li permet emetre signatures de nivell qualificat (QES, el nivell més alt). La seva oferta és fonamentalment B2B: s'integra dins de sistemes de tercers (ERP, CRM, gestors documentals) mitjançant una API REST i/o *iframe*. El seu cost és dinàmic segons la previsió anual de signatures (en desenes de milers per al primer tram) i el nivell d'aquestes. És una de les solucions de referència en el sector sanitari espanyol.
- **Signaturit** [16]: plataforma SaaS d'origen espanyol, també orientada al mercat B2B, que cobreix tots els nivells de signatura (simple, avançada i qualificada). El seu cost és per usuari, amb mòduls addicionals de pagament per a la verificació d'identitat, l'enviament per SMS i l'accés a l'API per a integracions.

- **Adobe Acrobat Sign** [17]: solució de signatura del gran ecosistema d'Adobe, molt famosa a escala internacional i molt integrada amb el seu entorn de documents. Com que la signatura no és el principal objectiu d'Adobe, sinó que és un afegit a l'entorn de documents, no han treballat en matèria de certificats propis, sinó que treballen amb prestadors qualificats de tercers per a oferir signatura qualificada. El model és de subscripció amb un cost addicional per cada signatura qualificada emesa. Està orientat tant a l'ús individual com a l'empresarial, i més aviat es podria dir que és un intermediari entre un proveïdor real de certificats digitals i un client de signatura.

LSigner no pretén competir comercialment amb aquestes plataformes, sinó demostrar que és possible construir, des de zero i amb tecnologies populars, una base tècnica sòlida de signatura electrònica de nivell professional.

2.2 Viabilitat tècnica

La viabilitat tècnica del projecte queda garantida per tres factors principals: la tecnologia disponible, un model de desplegament assequible i l'experiència prèvia en desenvolupaments similars.

El desenvolupament de programari web professional és un terreny molt madur i sobradament resolt. Existeixen diversos *frameworks* de *backend* i de *frontend*, tots ells robustos, de codi obert, i capaços d'implementar un producte professional de qualsevol tipus. A més a més, per a les necessitats específiques del projecte, com ara la criptografia o les utilitats de propòsit general (*stream* de binaris, per exemple), hi ha també moltes biblioteques consolidades, independents del *framework*, que cobreixen tot el que aquest projecte necessita. Fins i tot, hi ha *frameworks* que ja incorporen de manera nativa un ampli conjunt de biblioteques de caràcter general per cobrir la majoria de necessitats.

Pel que fa al desplegament, el mercat ofereix multitud de solucions de tipus *click to deploy* (Netlify o Vercel en són exemples) que abstrueixen tota l'arquitectura de desplegament i la infraestructura al darrere i permeten desplegar qualsevol projecte de manera molt ràpida i senzilla. Ara bé, precisament aquest no és el model que es busca, ja que un dels principals objectius del projecte és tenir control total d'aquest procés. Per tal que aquest objectiu sigui viable, n'hi ha prou amb tenir un proveïdor de servidor virtual KVM, un mercat amb una àmplia oferta molt fiable a un cost justificable. Afegit a això, la contenidorització amb Docker permet empaquetar i desplegar qualsevol servei de manera reproducible, i les eines que faciliten la coordinació dels contenidors i els desplegaments (amb serveis afegits com la renovació automàtica de certificats TLS) cobreixen la resta de necessitats sense haver de recórrer a serveis *cloud* costosos.

El tercer factor és l'experiència prèvia. La manca d'experiència en un *framework* web concret no faria inviable el projecte: la formació d'un enginyer de programari és, per definició, independent d'una tecnologia concreta, i aquesta sempre es pot aprendre. Ara bé, sí que suposaria una fricció important. En aquest cas, l'experiència professional prèvia amb la mateixa pila tecnològica del projecte, juntament amb el coneixement criptogràfic adquirit en l'antecedent de *blockchain*, redueix notablement aquesta fricció. Aquesta experiència prèvia es detalla al Capítol 5.

2.3 Viabilitat econòmica

Des del punt de vista econòmic, cal considerar diversos factors per avaluar el cost total del desenvolupament, més enllà dels purament tècnics. Un dels components econòmics per excel·lència en qualsevol projecte és el cost de les llicències de programari (tot i que en els darrers anys el model de negoci ha migrat a subscripcions); en aquest projecte, però, gairebé tota la pila tecnològica és de codi obert (*frameworks*, base de dades, biblioteques i sistema operatiu), de manera que aquest cost és pràcticament nul. El pes principal del pressupost recau, doncs, en el cost humà: com que el projecte el desenvolupa un únic desenvolupador, el factor determinant és el cost de les seves hores de dedicació, la descomposició detallada de les quals es presenta al capítol de planificació.

Ara bé, un producte com aquest no només té costos de desenvolupament, és clar, sinó que també té costos recurrents (subscripcions) d'infraestructura i de proveïdors de serveis externs: l'allotjament en un servidor (VPS), el domini, el servidor de correu electrònic i, en general, els serveis de tercers que la plataforma pugui necessitar (com ara l'enviament de correus o d'SMS). A l'escala d'aquest projecte aquests costos són molt reduïts, però a mesura que la plataforma creixés (més usuaris, més signatures i, per tant, més missatges enviats) aquests costos escalarien de manera proporcional, igual que el cost d'escalar horitzontalment els serveis. En el marc d'aquest projecte, però, aquest no ha estat el cas.

La Taula 2.1 recull el pressupost estimat del projecte, calculat per a un únic desenvolupador, suposant una experiència d'uns divuit mesos, per tant un perfil júnior, amb un cost per a l'empresa de 20 € l'hora.

TAULA 2.1: Pressupost estimat del projecte.

Concepte	Cost
RRHH (desenvolupament, 484 h)	9.680 €
Amortització de l'equip	156 €
VPS (Hostinger)	300 €
Domini (lsigner.com)	~14 €
Correu electrònic	~55 €
GitHub	~55 €
Claude Code	~92 €
Llicència del sistema operatiu (Linux)	0 €
Compilació i CI/CD (GitHub Actions)	0 €
Total	≈ 10.352 €

Així doncs, sumant el cost de personal, l'amortització de l'equip i les despeses d'infraestructura i de serveis externs, el cost total estimat del projecte se situa al voltant dels 10.352 €, del qual la immensa majoria correspon a les hores de desenvolupament. Aquesta xifra confirma que, tret del cost en hores de desenvolupament, el projecte és econòmicament molt assequible, precisament perquè es fonamentarà en tecnologies obertes i una infraestructura de cost reduït.

2.4 Viabilitat legal

La signatura electrònica és un àmbit fortament regulat, així que cal comprovar que es coneix la normativa que s'hi aplica, que es pot assumir i que encaixa amb l'abast proposat. S'hi identifiquen quatre marcs rellevants.

En primer lloc, el Reglament (UE) 910/2014 (eIDAS) [1], que regula la signatura electrònica i distingeix tres nivells de signatura: simple, avançada i qualificada (SES, AES i QES, respectivament), descrits amb més detall a la Secció 5.2. Els nivells avançat i qualificat exigeixen mesures com ara l'ús de certificats qualificats emesos per proveïdors acreditats i dispositius segurs de creació de signatura. L'obtenció d'aquests certificats comporta uns costos inassumibles per a un projecte d'aquest tipus. Aquesta anàlisi, doncs, determina una decisió clau: LSigner s'emmarca únicament en la signatura electrònica simple (SES), reforçada, però, amb evidències orientades al no-repudi. Aquesta limitació fa que el projecte sigui legalment viable sense renunciar a la qualitat tècnica. La signatura avançada o qualificada queda, doncs, com a línia de treball futur, condicionada a l'existència d'un model de negoci real que en justifiqui la inversió (Capítol 12).

En segon lloc, el Reglament (UE) 2016/679 (RGPD) [13], ja que el sistema tracta dades personals (identitat dels usuaris registrats o signants públics, correus electrònics, números de telèfon, evidències de signatura, etc.). El compliment d'aquest reglament és viable aplicant principis de privacitat des del disseny i de l'arquitectura: minimitzar les dades, controlar l'accés i tenir polítiques de conservació definides. En la pràctica, això es tradueix en el xifratge de les dades sensibles, la limitació del seu accés i unes polítiques de conservació conformes al RGPD.

En tercer lloc, la Llei 34/2002, de serveis de la societat de la informació i de comerç electrònic (LSSICE) [18], aplicable a una plataforma web que dona un servei en línia, i que imposa obligacions d'informació i transparència cap a l'usuari. En el cas d'LSigner, la plataforma haurà de disposar del conjunt de textos legals necessaris, consultables per qualsevol usuari en qualsevol moment, així com d'informació sobre l'ús de galetes i la gestió de dades i usuaris.

Finalment, l'Esquema Nacional de Seguretat (ENS) [14]: tot i que el seu compliment només és obligatori en el sector públic, els seus principis es prenen com a referència de bones pràctiques de seguretat. Les certificacions ISO de seguretat de la informació [19] es descarten pel seu elevat cost i per la manca de sentit que tenen per a un projecte acadèmic. El compliment concret de cada marc es detalla al Capítol 10.

2.5 Viabilitat ètica i ús d'IA

El desenvolupament d'aquest projecte preveu fer ús d'eines d'intel·ligència artificial com a suport, un ús que es detalla al capítol de metodologia. Des del punt de vista ètic, això planteja dues condicions. D'una banda, la integritat acadèmica: tot el treball assistit per IA s'ha de declarar de manera transparent, i les decisions d'arquitectura, la lògica de negoci i el disseny del sistema han de ser d'autoria pròpia, emprant la IA com a eina de suport i mai com a substitut del criteri d'enginyeria. De l'altra, el tractament de dades: les eines d'IA només s'han d'utilitzar sobre codi i documentació del projecte, sense introduir-hi dades personals reals de tercers.

Des de la perspectiva ètica del producte, LSigner gestionarà documents que poden contenir informació sensible, cosa que el disseny ha d'incorporar mitjançant el xifratge de les dades sensibles, el control d'accés per defecte i la minimització de dades, en línia amb el principi de privacitat del RGPD. A més, es planteja que la confiança no recaigui únicament en el proveïdor: cada evidència ha de ser verificable de manera independent gràcies a la signatura criptogràfica i a la traçabilitat auditable de les accions.

Finalment, dos principis més completen el desenvolupament responsable del producte. Pel que fa a la inclusió, la interfície ha de ser accessible, recolzant-se en llibreries de components que segueixen les pautes WAI-ARIA [20]. Pel que fa a la sostenibilitat, desplegar tot el sistema sobre un únic servidor virtual modest mitjançant contenidors evita el sobredimensionament de la infraestructura i en redueix el consum energètic associat.

2.6 Anàlisi de riscos

Els principals riscos del projecte són els relatius a la ciberseguretat. Se n'identifiquen els següents:

- Accés no autoritzat a documents
- Manipulació de signatures o evidències
- Filtració de credencials
- Atacs a la infraestructura de desplegament

Aquests riscos es preveu mitigar-los mitjançant un conjunt de mesures de seguretat, que es detallaran als capítols corresponents:

- Autenticació per defecte
- Signatura criptogràfica verificable
- Gestió de secrets fora del codi
- Aïllament de xarxa de la base de dades
- Anàlisi estàtica de seguretat en la integració contínua

Un segon grup de riscos és el de la planificació, usual en un projecte individual i ambiciós: la possible infravaloració del temps necessari. Aquest risc es preveu gestionar mitjançant una priorització contínua de l'abast, tal com es detalla al capítol de planificació.

Finalment, i amb una rellevància molt menor atès que el projecte no està obert a un públic client real, cal esmentar el risc de disponibilitat i pèrdua de dades. En un desplegament sobre un únic servidor virtual, aquest esdevé un punt únic de fallada; ara bé, el seu impacte és inferior i no es considera un gran risc. Tot i així, es preveu una mitigació bàsica mitjançant còpies de seguretat periòdiques de la base de dades i del servidor, i es deixa una arquitectura d'alta disponibilitat amb redundància com a possible millora futura.

2.7 Conclusió

L'estudi conclou que el projecte és viable en tots els punts analitzats (tècnic, econòmic, legal i ètic), tot i que és interessant fixar-se en els dos extrems de la balança.

El factor que més pesa en contra és, alhora, de naturalesa legal i econòmica: assolir els nivells de signatura avançada o qualificada exigiria certificats emesos per prestadors acreditats i dispositius segurs de creació de signatura, amb uns costos de certificació i auditoria senzillament inassumibles per a un projecte individual i de caràcter acadèmic. A això s'hi afegeix el repte de desenvolupar un projecte ambiciós dut a terme per una sola persona, en què la dificultat principal no és la tecnologia, sinó assolir el nivell de qualitat i robustesa proposat.

En canvi, el que més pesa a favor és que les tecnologies necessàries ja existeixen, són obertes i pràcticament gratuïtes, que la infraestructura per posar-les en marxa és econòmica i que, a més, es parteix d'una experiència prèvia rellevant. No hi ha cap barrera tècnica que no es pugui superar, el cost es concentra gairebé tot en les hores de feina i, un cop acotat l'abast, complir amb el marc legal és del tot assumible.

I és justament el fet de centrar el projecte en la signatura electrònica simple (SES), reforçada amb evidències orientades al no-repudi, el que converteix un problema que podria ser inviable en un projecte del tot factible. Amb aquest abast, tots els riscos i obstacles o bé desapareixen o bé es redueixen a un nivell controlable amb les mesures comentades anteriorment. En conclusió, el projecte no només és viable, sinó que és una bona oportunitat per demostrar que es pot construir, des de zero i amb pocs recursos, una plataforma de signatura electrònica amb qualitat professional.

Capítol 3

Metodologia

3.1 Metodologia de treball

El desenvolupament d'aquest projecte ha seguit una metodologia àgil adaptada, ja que ha estat dut a terme per una sola persona. Les metodologies àgils més esteses, com ara Scrum, estan pensades per a equips amb rols diferenciats, i aplicar-les al peu de la lletra en un projecte individual no tindria sentit. Per això, en comptes de seguir cap metodologia de manera estricta, se n'han adoptat els principis fonamentals (iteracions curtes, priorització i revisió contínua) i s'han aplicat amb un flux de treball de tipus Kanban. El treball s'ha organitzat en iteracions (*sprints*) de dues setmanes. Al final de cada una, es revisava amb el tutor del projecte cada canvi i avenç: una revisió periòdica que ha fet la funció d'un *sprint review*, com a validació externa i punt de retroalimentació, i que en ocasions ajudava a valorar les prioritats de la iteració següent. Les tasques es van gestionar amb GitHub Projects (alternativa a eines com Jira o YouTrack, integrada amb GitHub): cada tasca es registrava com un *issue* (una funcionalitat nova o un *bug* per arreglar) amb un identificador únic que també servia per anomenar la branca de treball associada (vegeu la Secció 3.2), de manera que en tot moment hi havia una connexió entre la tasca, la branca de codi i el *Pull Request*.

3.1.1 Definició adaptativa de l'abast

Un tret característic d'aquesta metodologia adaptada ha estat la redefinició constant de l'abast del projecte. Els requisits inicials es van deixar deliberadament difusos (la signatura de documents, els bloqueigs d'accés, la gestió de contactes i una possible orientació B2B): això permetia tenir una idea general clara del producte sense haver de tancar tots els requisits d'una sola vegada, i s'han anat refinant o redefinint a cada *sprint*. Aquesta manera de treballar encaixa amb la filosofia àgil: en comptes de definir-ho tot a l'inici, com faria un model *waterfall*, l'abast s'ha tractat de forma orientativa i revisable. Com a conseqüència, algunes tasques amb prioritat més baixa no es van arribar a implementar mai, o van quedar a mig fer: la migració dels fitxers binaris de la base de dades a un *bucket* d'S3, la pàgina de seguretat del *frontend* o la incorporació de multifactor o claus personals de signatura. Lluny de ser un defecte, això forma part del funcionament normal d'una metodologia àgil, en què cada *sprint* busca generar el màxim valor amb els recursos disponibles. Els canvis concrets respecte de la previsió inicial es detallen al Capítol 11.

3.2 Flux de treball i control de versions

El control de versions s'ha gestionat amb Git, amb una adaptació del model Git-Flow [21] de branques. El codi s'organitza en tres branques (**dev**, **uat** i **pro**), que corresponen als entorns de desenvolupament, preproducció i producció, respectivament (paradoxalment, se sol anomenar «entorn de producció» la branca **pro**, però tots tres entorns solen estar en producció, és a dir, desplegats en entorns reals). Cada tasca es desenvolupa en una branca de *feature* independent, amb la convenció de nomenclatura **LSB-<issue>-<TIPUS>/<descripció>** per al *backend* i **LSF-<issue>-<TIPUS>/<descripció>** per al *frontend*: el prefix (LSB o LSF) distingeix el repositori (*backend* o *frontend*), el número enllaça amb l'*issue* de GitHub Projects i el tipus indica la mena de canvi (*feature*, *fix*, *refactor*, etc.). Les branques tenen un seguit de regles de protecció (*branch protection rules*) que fan que cap canvi hi arribi directament: tot s'integra mitjançant *Pull Requests*, que fan de punt de control. Aquestes regles imposen dues condicions per poder fusionar: una revisió i aprovació manual del codi, i que totes les comprovacions automàtiques d'integració contínua (descrites a la secció següent) passin correctament. Si qualsevol de les dues no es compleix, la fusió queda bloquejada. La promoció entre entorns (**dev** -> **uat** -> **pro**) també es fa sempre via *Pull Request*, mai amb *push* directe, de manera que cap canvi arriba a producció sense haver superat les validacions de tots els entorns previs.

3.3 Cicle de desenvolupament

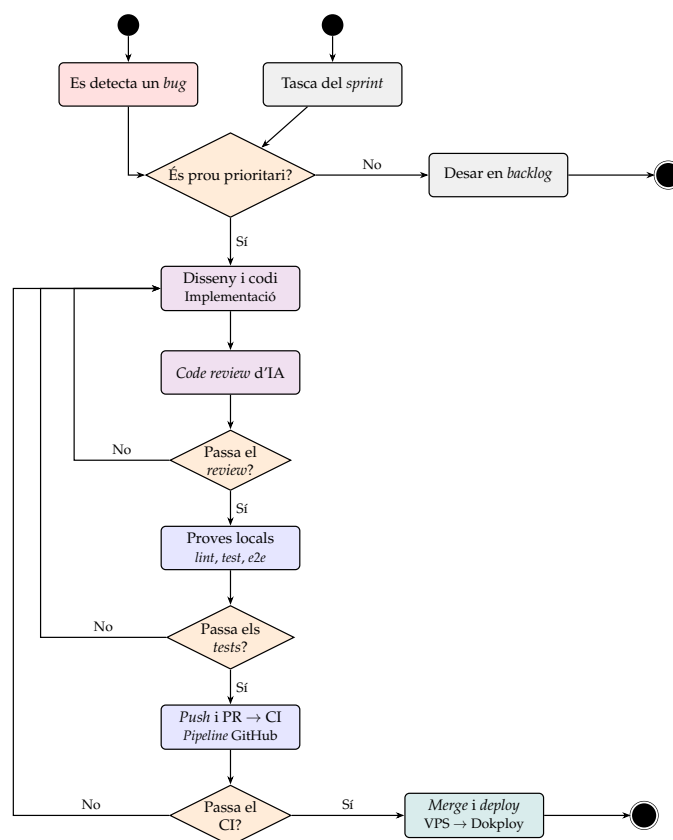


FIGURA 3.1: Cicle de desenvolupament i desplegament continu d'L-Signer. El flux és idèntic per al *backend* i el *frontend*.

La Figura 3.1 resumeix el cicle de desenvolupament complet, des de l'origen d'una tasca fins al desplegament a producció. *Backend* i *frontend* comparteixen la mateixa metodologia i estructura de *pipelines*, i només difereixen en alguns *tests* concrets. El cicle s'inicia de dues maneres: la detecció d'un error (*bug*) o la selecció d'una tasca planificada i prioritzada a l'*sprint* actual. En tots dos casos, la tasca passa primer per un filtre de prioritat: si no és prou prioritària, es desa al *backlog* per a una iteració futura; si ho és, entra al bucle de desenvolupament. Aquest bucle està pensat perquè tot el codi que arriba a producció tingui la mínima probabilitat de fallar. Comença amb el disseny i la implementació del canvi, i segueix amb una revisió de codi (*Code review*). Si la revisió detecta problemes, es torna a la implementació; si els supera, el canvi passa a la bateria de proves locals (format, *linting*, proves unitàries i *end-to-end*, i comprovació de tipus i compilació). Un altre cop, si les proves fallen, es torna enrere a la fase de codi. Un cop superades les proves locals, el canvi es puja al repositori i es crea un *Pull Request*, que dispara el *pipeline* d'integració contínua. Aquest *pipeline* torna a executar les comprovacions de manera automàtica i n'hi afegeix de seguretat, com ara l'anàlisi estàtica de codi (SAST) i l'auditoria de dependències. Només quan passen totes, el canvi es pot fusionar, i és llavors quan el servidor de producció detecta el canvi i en fa el desplegament automàtic. La intensitat de les comprovacions és gradual: els *Pull Requests* n'executen un subconjunt lleuger per no alentir el desenvolupament, mentre que les fusions i les promocions d'entorn les executen totes (Figura 3.2). El detall tècnic de cada *pipeline* es descriu al Capítol 9.

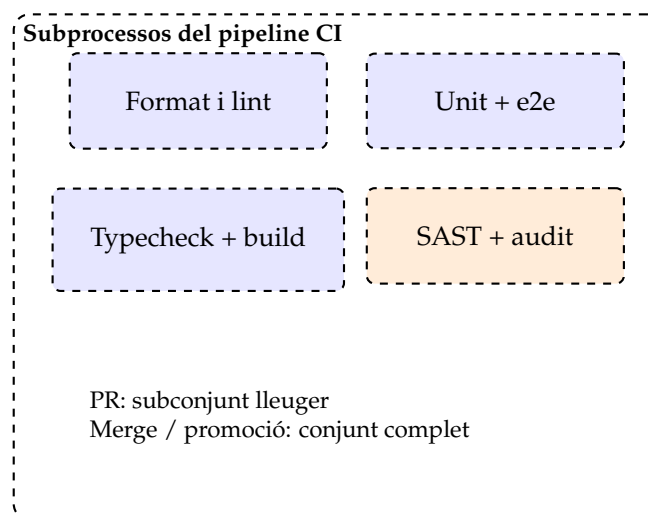


FIGURA 3.2: Comprovacions executades pel *pipeline* d'integració contínua. La intensitat és gradual segons el tipus (*Pull Request*, fusió o promoció d'entorn).

3.4 Ús d'eines d'intel·ligència artificial

En aquesta secció es declara de manera explícita l'ús d'eines d'intel·ligència artificial generativa durant l'elaboració del projecte. Aquestes eines s'han fet servir com a suport en diverses fases, sempre mantenint l'autoria pròpia de les decisions d'enginyeria, de l'arquitectura i de la lògica del sistema, tal com dicta la normativa. **En la generació de codi.** L'ús d'IA en el codi s'ha concentrat en dos àmbits. D'una banda, com a suport puntual per configurar biblioteques o funcionalitats més específiques

del *framework* que no coneixia prèviament (per exemple, la configuració de certs *pipes* de NestJS, de *middlewares* o d'aspectes del renderitzat de Next.js). Aquí encaixa molt bé l'ús d'IA, ja que soluciona el desconeixement d'eines específiques i no s'hi delega cap decisió d'arquitectura o semblant. D'altra banda, en el disseny visual del *frontend*: a partir d'una estructura visual i un mode de navegació per la plataforma que vaig decidir personalment, la IA va ajudar a derivar la paleta de colors i la maquetació de la pàgina principal, que després vaig fer servir de base per a la resta de pantalles. La lògica de negoci, les entitats, els serveis, els controladors i tota la lògica del *frontend* són d'autoria pròpia. **En la revisió de codi.** La IA ha tingut un paper molt important com a revisora. Al principi, la revisió de codi es duia a terme a través del revisor automàtic de GitHub, que s'activa cada cop que arriba un *Pull Request* i analitza els canvis en els fitxers involucrats. Aquesta revisió oferia *feedback* directe, que jo valorava per decidir si calia aplicar-lo. Més endavant, però, aquest procés es va integrar en un *hook* de *pre-commit* amb un agent de revisió. Això vol dir que la revisió de codi era més ràpida i immediata, ja que no hi havia el retard de GitHub a l'hora de preparar el seu propi entorn per revisar. Aquest ha estat un ús especialment útil per detectar problemes aviat i mantenir la qualitat del codi. **En l'e-laboració de la memòria.** S'ha fet servir la IA per ajudar a estructurar el document i revisar-ne l'estil, i també com a interlocutora per contrastar i argumentar decisions tècniques. Ara bé, la redacció, el contingut tècnic i totes les decisions del projecte són d'autoria pròpia. En tots els casos declarats, les eines d'IA s'han fet servir només sobre codi i documentació del mateix projecte, sense introduir-hi dades personals reals de tercers, tal com s'estableix a l'anàlisi de viabilitat ètica (Secció 2.5).

Capítol 4

Planificació

La planificació estableix l'estratègia seguida per assolir els objectius plantejats al Capítol 1. En un projecte individual i de llarga durada com aquest, la planificació no ha estat un cronograma tancat i immutable, sinó una guia revisable i coherent amb la metodologia àgil adaptada que s'ha descrit al capítol anterior. A més a més, per damunt de les iteracions curtes (*sprints*), el desenvolupament del projecte s'ha estructurat de manera molt clara en quatre grans blocs, o paquets de treball.

Aquest capítol descriu la planificació en dos nivells de detall: d'una banda, els paquets de treball, que agrupen el projecte en grans temes; i de l'altra, les iteracions de dues setmanes que s'han dut a terme dins de cada paquet i de forma recurrent. El capítol tanca amb el cronograma (un diagrama de Gantt), que mostra com les tasques es distribueixen en el temps al llarg dels prop de cinc mesos de desenvolupament.

4.1 Estratègia de planificació

L'enfocament de la planificació ha estat el mateix que el de la metodologia: prioritzar un desenvolupament adaptable segons l'avaluació constant dels requisits. En comptes de definir per endavant un pla detallat amb totes les tasques i la seva durada exacta, la planificació s'ha plantejat en dos nivells complementaris.

El nivell superior és el dels **paquets de treball**: quatre blocs temàtics amplis (*backend*, infraestructura i DevOps, *frontend*, i iteració final i memòria) que marquen la direcció general del projecte i que es corresponen, aproximadament, amb els cinc mesos de desenvolupament. Aquests paquets no s'han concebut com a fases tancades i seqüencials (hi ha hagut solapaments i retorns a fases anteriors), sinó com a èmfasis: en cada moment hi ha hagut un focus principal de treball, encara que això no ha impedit tocar altres parts quan ha calgut.

El nivell inferior és el de les **iteracions**, de dues setmanes cadascuna, que són la unitat de treball real. És dins de cada iteració que s'han concretat les tasques, s'han prioritzat i s'han revisat amb el tutor al final del cicle, tal com s'ha descrit al Capítol 3.

Una decisió important de planificació va ser la priorització de la infraestructura i el desplegament (DevOps), que es van abordar *abans* que el *frontend*, i no al final del projecte. Va ser una decisió deliberada. En muntar tota la cadena de desplegament sobre el *backend*, cada avenç posterior ja es podia provar en un entorn real, en comptes de deixar la integració i el desplegament per a l'últim moment, que és una font habitual de riscos i de feina acumulada.

4.2 Paquets de treball

Cada paquet agrupa un conjunt de tasques relacionades, amb una estimació orientativa de la dedicació en hores; el desglossament de cada paquet es recull a les Taules 4.1–4.4. El cost de cada tasca es calcula a partir del cost per hora de 20 € definit a l'estudi econòmic (Capítol 2). El detall tècnic de la feina de cada paquet es desenvolupa als capítols del nucli d'enginyeria d'aquesta memòria (anàlisi, disseny i implementació).

4.2.1 PT1: Backend

TAULA 4.1: Paquet PT1 (*backend*): tasques, dedicació estimada i cost.

Tasca	Descripció	Hores	Cost
Configuració inicial	Estructura modular de NestJS i configuració base (entorn, TypeORM, base de dades)	6	120 €
Autenticació	Registre, <i>login</i> , JWT, rotació de <i>refresh tokens</i> i <i>guards</i> globals	12	240 €
Usuaris	Entitat d'usuari, DTO i controlador i servei (registre, perfil, actualització)	6	120 €
Contactes	Entitat, CRUD i cerca	5	100 €
Documents	Entitat, versionat, estats i CRUD	10	200 €
Destinataris	Estats dels destinataris i relació amb els documents	6	120 €
Nucli de signatura	Claus Ed25519, <i>payload</i> canònic i generació i persistència d'artefactes	14	280 €
Cadena de custòdia	Esdeveniments <i>append-only</i> i evidència forense	8	160 €
OTP	<i>Challenges</i> , <i>hashing</i> amb sal, TTL, control d'intents i comparació <i>timing-safe</i>	9	180 €
Verificació	<i>Endpoint</i> públic de verificació independent de signatura	4	80 €
Bloqueig de documents	Contrasenya d'accés opcional per document	5	100 €
Sessions públiques	<i>Guard</i> , <i>bootstrap</i> i <i>cookie</i> HttpOnly per a enllaços públics	6	120 €
Migracions	Migracions de TypeORM i gestió de l'esquema global	4	80 €
Proves unitàries	Serveis i <i>pipelines</i> de signatura	10	200 €
Proves e2e	Proves d'extrem a extrem del <i>backend</i>	8	160 €
Documentació API	Documentació OpenAPI/Swagger dels controladors	3	60 €
Total		116	2.320 €

4.2.2 PT2: Infraestructura i DevOps

TAULA 4.2: Paquet PT2 (infraestructura i DevOps): tasques, dedicació estimada i cost.

Tasca	Descripció	Hores	Cost
VPS	Contractació i configuració inicial (SSH per clau, tallafof, <i>hardening</i> bàsic)	10	200 €
Domini i DNS	Domini <i>lsigner.com</i> i entrades DNS per subdomini	4	80 €
Dokploy	Instal·lació i configuració inicial	6	120 €
Reverse proxy i TLS	Traefik i TLS amb Let's Encrypt	6	120 €
Dockerfile backend	Imatge multietapa (base, dependències, <i>build</i> i producció)	10	200 €
CI: PR	Workflow de format, <i>lint</i> , <i>test</i> i <i>typecheck</i>	10	200 €
CI: dev/merge	Workflow amb Postgres i proves <i>e2e</i> reals	10	200 €
CI: promoció	Workflow de promoció a <i>uat</i> i <i>pro</i>	5	100 €
Seguretat al CI	SAST amb Semgrep i auditoria de dependències	6	120 €
Protecció de branques	Regles de protecció i flux GitFlow (<i>dev</i> , <i>uat</i> , <i>pro</i>)	4	80 €
docker-compose de dev	Backend i base de dades amb <i>bind-mounts</i> i <i>hot-reload</i>	8	160 €
Servei backend	Desplegament del <i>backend</i> a Dokploy	6	120 €
PostgreSQL	Contenidor a Dokploy, configuració i execució de migracions	5	100 €
Correu electrònic	Configuració del proveïdor de correu (SMTP) i registres DNS	4	80 €
Aïllament de la BD	Xarxa privada amb Tailscale i accés restringit a Postgres i pgAdmin	8	160 €
Dockerfile frontend	Imatge multietapa amb sortida <i>standalone</i>	9	180 €
Smoke test	Prova d'arrencada del contenidor de <i>frontend</i> al <i>pipeline</i>	3	60 €
Servei frontend	Desplegament del <i>frontend</i> a Dokploy	5	100 €
Emmagatzematge S3	S3 autoallotjat amb Garage (<i>buckets</i> i claus d'accés)	8	160 €
Còpies de seguretat	<i>Cron</i> nocturn, retenció de 7 còpies a S3 i verificació de restauració	6	120 €
Total		133	2.660 €

4.2.3 PT3: Frontend

TAULA 4.3: Paquet PT3 (*frontend*): tasques, dedicació estimada i cost.

Tasca	Descripció	Hores	Cost
Configuració Next.js	App Router, estructura de carpetes i configuració base	6	120 €
Sistema de disseny	Tokens de Tailwind (paleta clara i fosca) i tema de MUI	10	200 €

Tasca	Descripció	Hores	Cost
Disseny visual base	Derivació de la <i>homepage</i> i patrons de pantalla i <i>modals</i>	12	240 €
Client d'API	Client propi amb <i>fetch</i> : capa central, errors i <i>endpoints</i> tipats	10	200 €
Refresc de sessió	Refresc silencios amb bloqueig de concurrència i proves	8	160 €
Sessió i rutes	Context d'autenticació, <i>guard</i> i rutes protegides	9	180 €
Autenticació	Pàgines de <i>login</i> i registre amb validació de formularis	10	200 €
Layout de l'app	Navegació lateral, barra superior i peu	8	160 €
Homepage	Pàgina principal de la plataforma	8	160 €
Perfil i configuració	Perfil, configuració i esborrat del compte	8	160 €
Enviament de documents	Assistent per passos: pujada, metadades, destinataris i confirmació	14	280 €
Documents rebuts	Llista, detall i previsualització	12	240 €
Documents enviats	Llista de documents enviats	6	120 €
Signatura amb OTP	Flux de consentiment i verificació per OTP	12	240 €
Historial	Línia de temps immutable d'accions	6	120 €
Contactes	Pàgina, cerca i <i>autocomplete</i>	8	160 €
Pàgines legals	Pàgines amb <i>slug</i> dinàmic i integració d'anàlisi (Ackee)	5	100 €
Idiomes	Internacionalització (en/es/ca) i prova de paritat de claus	6	120 €
Proves frontend	563 proves amb Vitest	14	280 €
Components i pujada	Pujada amb <i>drag and drop</i> i components reutilitzables	5	100 €
Total		177	3.540 €

4.2.4 PT4: Iteració final i memòria

TAULA 4.4: Paquet PT4 (iteració final i memòria): tasques, dedicació estimada i cost.

Tasca	Descripció	Hores	Cost
Refinament	Correcció transversal dels defectes detectats	14	280 €
Poliment d'UI	Coherència visual entre pantalles	8	160 €
Redacció de la memòria	Redacció del document a partir de tota la documentació generada durant el procés	26	520 €
Revisió i defensa	Revisió final i preparació de la defensa	10	200 €
Total		58	1.160 €

4.3 Cronograma

La Figura 4.1 presenta el cronograma del projecte com un diagrama de Gantt amb el detall de totes les tasques, agrupades pels quatre paquets de treball, al llarg de les prop de vint-i-una setmanes de desenvolupament (del 15 de febrer al 20 de juliol), a un ritme aproximat de 23 hores setmanals. Les barres reflecteixen els solapaments ja comentats: les darreres tasques del *backend* s'encavalquen amb les d'infraestructura i DevOps (que arrenca a l'inici de març), i les darreres d'aquest, amb les primeres del *frontend*; la iteració final i la redacció de la memòria acompanyen el tram final.

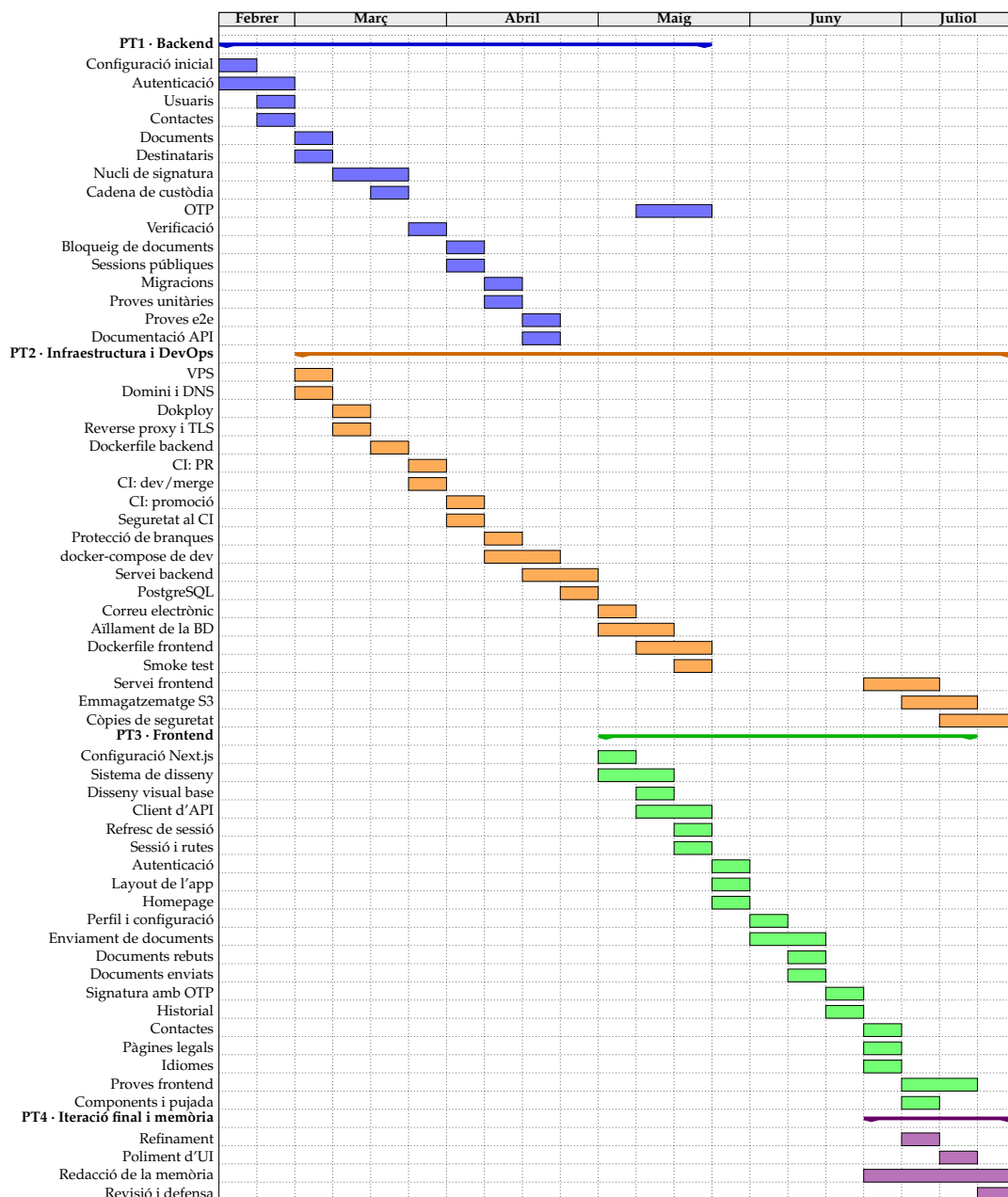


FIGURA 4.1: Cronograma del projecte (diagrama de Gantt) amb el detall de les tasques, agrupades per paquets de treball, del 15 de febrer al 20 de juliol.

Capítol 5

Marc de treball i conceptes previs

Abans d'entrar en el desenvolupament del projecte, aquest capítol dona context de tot el que s'ha dut a terme i introdueix els conceptes previs necessaris per comprendre els capítols següents. La primera part descriu el marc de treball (l'entorn, el punt de partida i els coneixements de què es partia); la segona defineix el vocabulari i els fonaments tècnics i legals sobre els quals es construeix la plataforma.

5.1 Marc de treball

A diferència dels projectes que es desenvolupen dins d'una empresa, una institució o un grup de recerca, aquest treball s'ha dut a terme de manera completament individual i partint de zero: no hi havia cap base de codi, cap infraestructura ni cap component previ sobre el qual construir. Tot el que es descriu en aquesta memòria s'ha dissenyat i implementat des de l'inici en el marc del projecte.

Ara bé, cal distingir l'origen conceptual de la propietat del projecte. La idea neix d'un context professional (l'experiència amb proveïdors de signatura en el sector *healthtech*, concretament en l'àmbit dels consentiments electrònics, i l'antecedent d'una prova de concepte basada en *blockchain*, tots dos descrits al Capítol 1), però LSigner és un projecte completament independent: no forma part de cap producte de l'empresa, no en reutilitza cap codi ni cap servei, i no hi ha cap violació de propietat intel·lectual. L'experiència professional ha aportat el coneixement i ha deixat entreveure la possibilitat d'una solució com aquesta en un mercat en pujada, però el projecte, el seu disseny i la seva implementació són d'autoria pròpia i exclusiva.

5.1.1 Persones involucrades

En tractar-se d'un projecte individual, les persones implicades es redueixen a dues. D'una banda, jo mateix, que he assumit tots els rols del desenvolupament: anàlisi, disseny, implementació, proves i desplegament. De l'altra, el tutor del projecte, que ha exercit una funció de validació externa i seguiment: en les revisions periòdiques al final de cada iteració (vegeu el Capítol 3) es posava en comú l'avenç i es reorientaven les prioritats quan calia. En les darreres fases del desenvolupament, una tercera persona validava puntualment el funcionament en producció.

5.1.2 Coneixements i experiència previs

El punt de partida en termes de coneixement no era el d'un estudiant i prou. A l'inici del projecte acumulava aproximadament catorze mesos d'experiència professional

en desenvolupament web, precisament amb la mateixa pila tecnològica emprada en aquest projecte tant al *backend* com al *frontend*, així com una base inicial en pràctiques de DevOps. A més, part del coneixement prové directament dels antecedents descrits al Capítol 1: del contacte amb proveïdors de signatura en el sector *healthtech* va sorgir la idea, i de la prova de concepte basada en *blockchain* en vaig extreure la noció de com generar i encadenar evidències verificables, que és precisament el nucli de la signatura electrònica.

Aquesta experiència prèvia és rellevant per dos motius. Primer, perquè redueix la fricció d'aprenentatge de les tecnologies base i permet centrar l'esforç en els reptes de disseny i de qualitat, més que no pas en la sintaxi o el funcionament elemental de les eines. Segon, perquè emmarca de manera honesta l'abast del que constitueix aprenentatge nou en aquest projecte: no tant els *frameworks* en si, sinó la seva aplicació a un domini real, l'assoliment d'un nivell de qualitat i robustesa de producció, i l'automatització completa del cicle de vida.

5.2 Conceptes previs

5.2.1 La signatura electrònica i el marc legal

Una **signatura electrònica** és un conjunt de dades en format electrònic que s'associen a un document i que permeten identificar el signant i garantir la seva conformitat amb el contingut. Convé distingir-la d'una signatura convencional digitalitzada, és a dir, la imatge d'una firma enganxada dins d'un PDF o un document escanejat. Aquesta darrera no aporta res des del punt de vista de la seguretat: no porta cap metadada ni cap informació associada, és un document com qualsevol altre i, si algú l'altera, no en queda cap rastre comprovable (només se'n percep el canvi visual). Una signatura electrònica ben construïda, en canvi, aporta garanties tècniques verificables.

Aquestes garanties són quatre. La **integritat**, que assegura que el document no s'ha alterat des del moment de la signatura. L'**autenticitat**, que vincula la signatura amb la identitat del signant. La **traçabilitat**, que és el registre verificable de quan i com s'ha produït cada acció. Finalment, el **no-repudi**, que és la impossibilitat que el signant negui posteriorment haver signat; aquesta darrera garantia, però, només s'assoleix plenament en els nivells de signatura més alts, que vinculen la signatura a una identitat certificada de manera inequívoca.

A escala europea, la signatura electrònica està regulada pel Reglament (UE) 910/2014, conegut com a eIDAS [1], que distingeix tres nivells de garantia:

- La **signatura electrònica simple (SES)** és la forma més bàsica: qualsevol dada electrònica associada a la voluntat de signar. Ofereix les garanties tècniques bàsiques, però no exigeix una identificació certificada del signant.
- La **signatura electrònica avançada (AES)** afegeix requisits: ha d'estar vinculada de manera única al signant, permetre'n la identificació, i crear-se amb mitjans sota el control exclusiu del signant, de manera que qualsevol alteració posterior sigui detectable. Requereix, a la pràctica, claus o certificats individuals per signant.

- La **signatura electrònica qualificada (QES)** és la de nivell més alt i té la mateixa validesa jurídica que una signatura manuscrita. Exigeix un certificat qualificat emès per un prestador de serveis de confiança acreditat i un dispositiu segur de creació de signatura.

Com s'ha justificat a l'estudi de viabilitat (Capítol 2), aquest projecte s'emmarca en el nivell SES, per les raons de cost i abast que s'hi detallen.

El valor d'una signatura electrònica, fins i tot en el nivell SES, rau en les **evidències** que es generen al voltant de l'acte de signar: el registre de qui signa i de com se n'ha verificat la identitat (per exemple, mitjançant un codi d'un sol ús, OTP, enviat per correu o SMS), quan es produeix la signatura, l'empremta del document en aquell moment i tota la traça d'accions associada. Són aquestes evidències, i no la imatge de la firma, les que permeten sostenir la integritat, l'autenticitat i la traçabilitat de manera comprovable a posteriori. Aquesta és la diferència de fons respecte de la simple imatge d'una firma: no es tracta de reproduir l'aparença de signar, sinó de deixar-ne proves verificables.

5.2.2 Fonaments criptogràfics

La solidesa tècnica d'una signatura electrònica es recolza en dos fonaments criptogràfics: les funcions de *hash* i la criptografia asimètrica.

Una **funció de hash** (com SHA-256 [22]) és una funció que, a partir de qualsevol entrada (per exemple, un fitxer o qualsevol seqüència de dades), produeix una sortida de mida fixa anomenada resum o empremta (*hash*). Té dues propietats clau: és determinista (la mateixa entrada produeix sempre la mateixa empremta) i és molt difícil trobar dues entrades diferents amb la mateixa empremta o reconstruir l'entrada a partir de l'empremta. Això la fa ideal per detectar alteracions: si un sol bit del document canvia, la seva empremta canvia completament.

Convé matisar per què es parla de «dificultat» i no d'impossibilitat absoluta. Com que l'empremta té una mida fixa, el conjunt d'empremtes possibles és finit, mentre que el d'entrades possibles no ho és; per tant, matemàticament és inevitable que existeixin entrades diferents que comparteixin la mateixa empremta (les anomenades col·lisions). El que fa segura la funció no és, doncs, que no hi hagi col·lisions, sinó que trobar-ne una de manera intencionada és computacionalment inviable: caldria provar una quantitat d'entrades tan enorme que resulta inabastable en un temps raonable.

La **criptografia asimètrica** (o de clau pública) es basa en un parell de claus relacionades matemàticament: una **clau privada**, que es manté en secret, i una **clau pública**, que es pot distribuir lliurement. En l'àmbit de la signatura, el mecanisme és el següent: el signant genera la signatura a partir de les dades i de la seva clau privada, i qualsevol tercer pot verificar-la utilitzant únicament la clau pública corresponent.

La seguretat de tot plegat descansa en el fet que la relació entre les dues claus és d'un sol sentit. A partir de la clau privada és immediat obtenir la clau pública i generar signatures, però el camí invers (deduir la clau privada a partir de la clau pública o de les signatures que ha produït) és computacionalment inviable, igual que ho és invertir una funció de *hash*. Aquesta asimetria és el que dona nom al mètode i el que

el fa segur: per molt que es coneguin la clau pública i totes les signatures generades, la clau privada no es pot reconstruir.

D'aquí se'n deriven directament les dues garanties. Com que només qui té la clau privada pot haver generat una signatura vàlida, es garanteix l'**autenticitat**. I com que la signatura es calcula sobre l'empremta del contingut, qualsevol modificació posterior del document en canvia les dades i, per tant, també l'empremta: en verificar la signatura contra el document alterat la comprovació ja no quadra i es dona per invàlida. Per generar de nou una signatura vàlida sobre el document modificat caldria la clau privada del signant, que, com acabem de veure, és computacionalment inviable d'obtenir; això és el que garanteix la **integritat**.

Ed25519 [23] és un esquema concret de signatura digital de clau pública, modern i àmpliament adoptat, que és el que s'ha emprat en aquest projecte. Les raons de la seva elecció davant d'altres esquemes (com RSA o ECDSA) es discuteixen al Capítol 7.

5.2.3 Arquitectura d'aplicacions web

Una aplicació web moderna se sol estructurar en tres grans components. El **backend** és la part que s'executa al servidor: conté la lògica de negoci, gestiona les dades i n'exposa la funcionalitat. El **frontend** és la part que s'executa al navegador de l'usuari i amb la qual aquest interactua. I la **base de dades** és on es persisteixen les dades de manera duradora.

Internament, el **backend** no s'organitza com un bloc únic, sinó separant responsabilitats en capes. El patró clàssic és el **MVC** (*Model-View-Controller*), que separa les dades (model), la presentació (vista) i el control del flux (controlador). En aplicacions modernes com aquesta, on la vista viu al **frontend**, l'organització és una variant per capes: uns components reben les peticions i en validen l'entrada (controladors), uns altres contenen la lògica de negoci (serveis) i uns altres s'encarreguen de l'accés a les dades. El **backend** d'aquest projecte s'implementa amb NestJS [24], un *framework* de Node que empeny cap a aquesta organització mitjançant mòduls, controladors i serveis. Dividir responsabilitats així és beneficiós perquè cada capa té una única funció, cosa que fa el codi més fàcil d'entendre, de provar i de mantenir: es pot modificar una capa sense arrossegar les altres.

Per a l'accés a les dades s'utilitza un **ORM** (*Object-Relational Mapping*), en aquest cas TypeORM [25]. Un ORM fa de pont entre la base de dades relacional i el codi: mapa les taules a classes i les files a objectes, de manera que es treballa amb objectes del llenguatge en comptes d'escriure consultes SQL a mà. Això redueix errors, centralitza la definició de l'estructura de les dades i facilita els canvis d'esquema.

El model d'execució de Node.js, l'entorn sobre el qual funciona el **backend**, es basa en un únic fil principal governat per un bucle d'esdeveniments (*event loop*): en lloc de bloquejar-se esperant operacions lentes (accés a disc, xarxa o base de dades), les delega i continua atenent altres peticions mentrestant. Aquest model no bloquejant li permet gestionar molta concurrència de manera eficient sense necessitat d'un fil per petició. Per a les tasques intensives de càlcul, que sí que bloquejarien el bucle, Node ofereix a més els *worker threads*, que permeten executar feina en paral·lel en fils separats. Aquesta darrera funció, però, no ha calgut per a aquest projecte.

Pel que fa al *frontend*, aquest projecte es construeix amb Next.js [26], un *framework* basat en la biblioteca React [27]. React permet construir la interfície com un conjunt de components reutilitzables que s'actualitzen sols quan canvien les dades. Next.js hi afegeix l'estructura i les funcionalitats que a React li falten per muntar una aplicació completa: un sistema d'enrutament (quines pàgines corresponen a quines adreces), la possibilitat de generar part de les pàgines al servidor abans d'enviar-les al navegador (**SSR**, *Server-Side Rendering*), cosa que millora el temps de càrrega inicial i el posicionament als cercadors, i una sèrie d'optimitzacions automàtiques (empaquetat del codi, càrrega diferida i optimització d'imatges). En resum, aporta una base sòlida amb uns valors per defecte raonables i evita haver de muntar tota aquesta infraestructura des de zero, cosa que explica bona part de la seva adopció.

La comunicació entre *frontend* i *backend* es fa habitualment mitjançant una **API REST**. L'API forma part del mateix codi del *backend* i n'és la porta d'entrada: defineix quines operacions es poden demanar a la lògica de negoci i de quina manera s'han de demanar. Sense API, el *backend* no té cap via per rebre instruccions de l'exterior. En concret, una API REST exposa la funcionalitat del servidor a través d'operacions sobre recursos (crear, llegir, actualitzar i esborrar; CRUD, per les sigles en anglès) utilitzant el protocol HTTP: el client fa peticions i en rep respostes, normalment en format JSON.

Aquest disseny separa qui demana les coses de qui les executa. Poden conviure diversos *frontends* (una aplicació web, una de mòbil, un rellotge intel·ligent o un sistema com CarPlay) i tots accedeixen a la mateixa lògica cridant la mateixa API de la mateixa manera. El *backend* no necessita saber quin tipus de client li parla: només ha d'atendre les instruccions que li arriben a través de l'API.

Per accedir de manera controlada a la funcionalitat protegida, s'utilitzen mecanismes d'autenticació. Un dels més estesos és el **JWT** (*JSON Web Token*) [28]: un testimoni signat que el servidor emet quan un usuari s'autentica i que el client presenta en cada petició posterior per demostrar la seva identitat, sense que el servidor hagi de mantenir l'estat de la sessió.

5.2.4 Contenedors i desplegament

Un **contenedor** és una unitat de programari que empaqueta una aplicació juntament amb totes les seves dependències (biblioteques, configuració i entorn d'execució) i l'executa de forma aïllada de la resta del sistema. Tècnicament, un contenidor no és una màquina ni un sistema operatiu sencer: és un procés (o un grup de processos) que s'executa sobre el nucli de la màquina amfitriona, però al qual el sistema operatiu aplica una sèrie de restriccions que el fan creure que s'executa tot sol en una màquina dedicada. Aquestes restriccions són, bàsicament, de dos tipus: d'aïllament (el contenidor només veu els seus propis processos, fitxers i xarxa) i de recursos (se li limita quanta CPU, memòria o disc pot fer servir). El contenidor no porta el seu propi nucli (*kernel*), sinó que aprofita el de l'amfitrió, i és justament això el que el fa molt més lleuger que una màquina virtual, que sí que virtualitza el maquinari i hi instal·la un sistema operatiu complet.

El que s'executa dins d'un contenidor prové d'una **imatge**: una plantilla de només lectura, construïda per capes, que conté el sistema de fitxers i tot allò que necessita l'aplicació. Cada capa hi afegeix una part segons la funcionalitat (per exemple, primer el sistema base, després l'entorn d'execució i finalment l'aplicació), de manera

que es poden reaprofitar capes comunes entre imatges. **Docker** [11] és la tecnologia de contenidors més estesa.

El desplegament d'aquest projecte es fa sobre un **VPS** (*Virtual Private Server*): un servidor virtual, amb recursos dedicats (CPU, memòria i disc), llogat a un proveïdor i sobre el qual es té control total de la configuració. Un VPS és, en essència, una màquina virtual gestionada amb una tecnologia de virtualització com KVM (*Kernel-based Virtual Machine*), i això li dona uns quants avantatges pràctics: com que tot el servidor es representa amb una imatge del seu sistema operatiu, es pot clonar i traslladar a un altre amfitrió, se'n poden fer còpies de seguretat completes i es pot escalar assignant-li més recursos quan calgui. Davant dels serveis desplegats, un *reverse proxy* actua d'intermediari: rep totes les peticions entrants i les encamina cap al servei intern corresponent segons el domini, i s'encarrega també de la gestió del xifratge **TLS** [29] (el protocol que xifra la comunicació entre el client i el servidor, i que és el que hi ha darrere del cadenet d'HTTPS).

Tenir un VPS, però, és només el punt de partida: desplegar-hi una aplicació de manera professional implica moltes coses més. Cal mantenir els serveis en marxa i reiniciar-los si cauen, recollir-ne els *logs*, fer còpies de seguretat, gestionar els desplegaments de noves versions (idealment amb registre d'auditoria i possibilitat de *rollback* a una versió anterior si alguna cosa falla), administrar els certificats i els dominis, i orquestrar la composició entre els diferents serveis (*backend*, *frontend*, base de dades, etc.). Plataformes com Amazon Web Services (AWS) ofereixen tot això de manera molt completa i escalable, però amb una complexitat i un cost considerables. En aquest projecte s'ha optat per **Dokploy** [12]: una plataforma de desplegament autoallotjada, construïda sobre Docker, que ofereix aquestes mateixes capacitats (desplegaments amb *rollback*, *logs*, còpies de seguretat, certificats automàtics mitjançant Traefik [30] i composició de serveis) de manera molt més simple. És menys potent que les grans solucions comercials, però resulta ideal per aprendre i entendre de prop tot el cicle de desplegament.

5.2.5 Integració i desplegament continu (CI/CD)

La **integració contínua** (CI, *Continuous Integration*) és la pràctica d'integrar i validar els canvis de codi de manera freqüent i automàtica: cada vegada que s'incorpora un canvi, un sistema automatitzat executa un conjunt de comprovacions (compilació, proves, anàlisi de qualitat i de seguretat) per detectar problemes com més aviat millor. El **desplegament continu** (CD, *Continuous Deployment*) n'és l'extensió natural: un cop superades totes les comprovacions, el canvi es desplega automàticament a l'entorn corresponent.

Aquestes comprovacions són **limitants**: si alguna falla (per exemple, una prova o una anàlisi de seguretat), el procés s'atura i el canvi no arriba a integrar-se ni a desplegar-se. Aquest és precisament el mecanisme que evita que codi defectuós arribi a producció. A més, les comprovacions no s'executen a la màquina del desenvolupador, sinó en una màquina neta i independent que es crea de zero per a cada execució; així es reproduïx un entorn net i es garanteix que el projecte funciona per si sol, sense dependre de res que hi pugui haver instal·lat localment.

El conjunt d'aquestes comprovacions i passos automatitzats s'anomena *pipeline*. Un avantatge d'aquest enfocament és la flexibilitat: un *pipeline* no és una llista tancada

de passos, sinó una seqüència a la qual sempre se'n poden afegir de nous. S'hi poden incorporar més proves, anàlisis estàtiques de codi, cerques de vulnerabilitats, comprovacions de format o qualsevol altra validació que interessi, sense haver de tocar el codi de l'aplicació. En aquest projecte, els *pipelines* s'implementen amb **GitHub Actions** [10], la solució d'automatització integrada a la plataforma GitHub. El detall concret dels *pipelines* d'aquest projecte es descriu al Capítol 9.

5.2.6 Proves de programari

Les **proves** (*tests*) són sovint una part important del desenvolupament: codi automatitzat que comprova que una altra part del codi es comporta com s'espera. En el desenvolupament web el seu pes pot ser menor que en altres àmbits, perquè molts fluxos es poden comprovar fàcilment a mà des del navegador, però continuen sent valuoses per detectar regressions i per validar de manera repetible la lògica crítica.

Es distingeixen diversos tipus segons què cobreixen. Les **proves unitàries** validen peces aïllades de codi i sovint en substitueixen les dependències per versions simulades (*mocks*) per poder-les provar de manera controlada. Les **proves d'extrem a extrem** (**E2E**, *end-to-end*) es col·loquen a l'altre extrem: recreen l'ús real de l'aplicació simulant les accions d'un usuari sobre el sistema complet. En aquest projecte les proves s'integren dins el *pipeline* de CI, i el seu detall es tracta al Capítol 9.

Capítol 6

Requisits del sistema

Abans d'enumerar els requisits, convé identificar els actors del sistema, ja que molts requisits s'entenen millor en funció de qui els fa servir. Se'n distingeixen tres:

- **Usuari registrat.** És l'actor principal. Disposa d'un compte i pot gestionar els seus documents i contactes, enviar documents a tercers i signar o rebutjar els documents que rep. Actua, doncs, tant d'emissor com de signant.
- **Destinatari extern.** És una persona sense compte a la plataforma que rep un document per signar mitjançant un enllaç públic. La seva interacció es limita a accedir al document, verificar la seva identitat i signar-lo o rebutjar-lo.
- **Sistema.** Alguns requisits no els desencadena cap persona, sinó processos automàtics del propi sistema: la generació d'evidències criptogràfiques, les còpies de seguretat periòdiques o la purga de dades segons les polítiques de retenció.

6.1 Requisits funcionals

La Taula 6.1 recull els requisits funcionals del sistema. La columna d'estat indica si el requisit està completament o parcialment implementat, en coherència amb la definició adaptativa de l'abast.

TAULA 6.1: Requisits funcionals del sistema.

ID	Descripció	Prioritat	Estat
RF1	Gestió d'usuaris i autenticació. Registre, inici de sessió, gestió del perfil i eliminació del compte amb anonimització de les dades personals, amb autenticació basada en JWT i rotació de <i>refresh tokens</i> .	Alta	Implementat
RF2	Gestió de contactes. Alta, consulta, cerca i eliminació dels contactes de l'usuari.	Mitjana	Implementat
RF3	Gestió de documents. Pujada, versionat, control d'estats i cicle de vida complet del document.	Alta	Implementat

ID	Descripció	Prioritat	Estat
RF4	Enviament a destinataris. Enviament d'un document a un o més destinataris (registrats o externs), amb notificació per correu electrònic i possibilitat d'enviar recordatoris als destinataris pendents.	Alta	Implementat
RF5	Signatura, rebuig i revocació de documents. Acte de signar o rebutjar un document per part d'un destinatari, així com la possibilitat de revocar posteriorment una acció de signatura.	Alta	Implementat
RF6	Verificació de la identitat del signant (OTP). Codi d'un sol ús, enviat per correu electrònic, que el sistema exigeix com a segon factor abans de qualsevol operació criptogràfica.	Alta	Implementat
RF7	Generació d'evidències i signatura criptogràfica. Creació de l'artefacte de signatura Ed25519 sobre un <i>payload</i> canònic, amb l'empremta del document i la cadena de custòdia associada.	Alta	Implementat
RF8	Verificació independent. <i>Endpoint</i> públic que permet a qualsevol tercer verificar una signatura sense necessitat de compte ni de sessió a la plataforma.	Alta	Implementat
RF9	Historial d'accions. Línia de temps immutable de les accions realitzades sobre els documents.	Mitjana	Implementat
RF10	Bloqueig d'accés a documents. Contraseña d'accés opcional per document, establerta per l'emissor.	Baixa	Parcial
RF11	Accés de destinataris externs. Signatura mitjançant enllaç públic amb sessió temporal, sense necessitat de compte.	Mitjana	Parcial
RF12	Internacionalització. Interfície disponible en català, castellà i anglès.	Baixa	Implementat
RF13	Retenció i purga de dades. Procés automàtic que elimina periòdicament els registres que superen el període de retenció establert (12 mesos), en compliment de les polítiques de conservació.	Mitjana	Implementat

Cal fer un aclariment sobre els requisits marcats com a parcials. El RF10 (bloqueig de documents) està implementat al *backend* i parcialment integrat al *frontend*. El RF11 (accés de destinataris externs) té el flux complet implementat al *backend* (sessió pública temporal, *cookie* segura i el mateix procés de signatura que els usuaris registrats), mentre que la integració al *frontend* està per implementar. El detall del grau d'assoliment es tracta al Capítol 10.

Els casos d'ús del sistema, amb el seu diagrama UML i les fitxes dels més rellevants, es detallen al Capítol 8, on es vinculen amb l'anàlisi de cada requisit.

6.2 Requisits no funcionals

Els requisits no funcionals defineixen les qualitats transversals que ha de tenir el sistema. En un projecte l'objectiu del qual és, precisament, la qualitat d'enginyeria per damunt de la quantitat de funcionalitats (vegeu el Capítol 1), aquests requisits tenen un pes especialment rellevant. La Taula 6.2 els recull. A diferència dels funcionals, la columna d'estat n'expressa el grau d'assoliment (assolit o parcial), atès que es tracta de propietats transversals i contínues, i no de funcions discretes; el seu compliment detallat s'avalua al Capítol 10.

TAULA 6.2: Requisits no funcionals del sistema.

ID	Descripció	Prioritat	Estat
RNF1	Seguretat. Autenticació per defecte (<i>secure by default</i>), gestió de secrets fora del codi, aïllament de xarxa de la base de dades i anàlisi estàtica de seguretat (SAST) integrada al <i>pipeline</i> de CI.	Alta	Assolit
RNF2	Privadesa i compliment normatiu. Alineació amb el RGPD i amb els principis de l'ENS: minimització de dades, control d'accés i polítiques de conservació definides.	Alta	Assolit
RNF3	Qualitat i mantenibilitat. Arquitectura modular, aplicació de patrons de disseny i una cobertura de proves àmplia que faciliti l'evolució del codi.	Alta	Assolit
RNF4	Verificabilitat i no-repudi. Les evidències de signatura han de ser verificables per tercers de manera independent, sense dependre de la disponibilitat de la plataforma, dins els límits del nivell SES adoptat (Capítol 2).	Alta	Assolit
RNF5	Desplegabilitat i operació. Cicle de desplegament automatitzat, amb possibilitat de <i>rollback</i> , còpies de seguretat periòdiques i registre de <i>logs</i> .	Alta	Assolit
RNF6	Preparació per a l'escalabilitat. Arquitectura modular preparada per créixer, tot i que el projecte no ha requerit validar un escalat real.	Mitjana	Parcial
RNF7	Usabilitat i accessibilitat. Interfície moderna i clara, recolzada en components que segueixen les pautes d'accessibilitat WAI-ARIA.	Mitjana	Parcial

ID	Descripció	Prioritat	Estat
RNF8	Rendiment i temps de càrrega optimitzats. La interfície ha d'oferir temps de càrrega ràpids. Es valida al Capítol 10 mitjançant mètriques de càrrega mesurades sobre l'entorn de producció.	Mitjana	Assolit

La major part d'aquests requisits no funcionals es concreten en decisions de disseny, d'implementació i de desplegament que es desenvolupen als capítols corresponents (Capítols 8, 9 i 10), on també s'avalua el seu grau de compliment.

Capítol 7

Estudis i decisions

Aquest capítol recull les decisions tècniques del projecte i les seves justificacions: el maquinari, la pila de programari de cada capa i les eines de suport o biblioteques. Les tecnologies en si ja s'han presentat al Capítol 5; aquí no es torna a explicar què són, sinó per què es van triar davant de les seves alternatives.

7.1 Maquinari

El maquinari del projecte es redueix a dues peces. D'una banda, l'equip de desenvolupament personal: un ordinador portàtil amb Linux, suficient per a tot el cicle de treball local (desenvolupament, proves i execució de contenidors). De l'altra, el servidor de producció: un VPS de Hostinger sobre virtualització KVM, amb 2 vCPU, 8 GB de RAM i 100 GB de disc NVMe. La justificació és la ja exposada a l'estudi de viabilitat (Capítol 2): recursos suficients per executar tots els serveis de la plataforma, cost contingut i control total de la màquina.

7.2 Pila del *backend*

7.2.1 Node.js i NestJS

La primera gran decisió de la pila va ser el llenguatge i el *framework* del *backend*. Es va optar per Node.js amb NestJS [24], davant d'alternatives consolidades com Spring (Java) o Django (Python), per tres motius.

El primer, i determinant, és la coherència de llenguatge amb el *frontend*. En emprar TypeScript tant al *backend* (NestJS) com al *frontend* (Next.js), es treballa amb un únic llenguatge a tot el projecte. Això redueix la càrrega cognitiva de canviar de context i permet compartir tipus i models entre les dues capes.

El segon és l'estructura que imposa NestJS. A diferència de *frameworks* més minimalistes, NestJS estableix una arquitectura modular i per capes (mòduls, controladors i serveis) i incorpora de manera nativa la injecció de dependències. Això encaixa directament amb l'objectiu de qualitat i mantenibilitat del projecte: el *framework* guia cap a un codi ben organitzat.

El tercer és l'experiència prèvia amb aquesta pila (Capítol 5), que redueix la fricció d'aprenentatge i permet centrar l'esforç en el disseny i la qualitat.

7.2.2 TypeORM i la gestió de transaccions

Per a l'accés a dades es va triar TypeORM [25]. Ara bé, la decisió tècnicament més rellevant d'aquesta capa no va ser l'ORM en si, sinó com utilitzar-lo: dels dos enfocaments que ofereix (el patró *Repository*, de més alt nivell, i l'*EntityManager*, de més baix nivell), s'ha optat majoritàriament per l'*EntityManager*, i el motiu és la gestió de transaccions.

Moltes operacions del sistema (com signar un document) impliquen diverses escriptures que han de tenir èxit o fracassar totes alhora: crear l'artefacte de signatura, actualitzar l'estat del destinatari i registrar l'esdeveniment d'auditoria. Una transacció garanteix aquesta atomicitat: o es confirmen tots els canvis, o no se'n confirma cap (*rollback*), de manera que la base de dades mai queda en un estat inconsistent. L'*EntityManager* permet executar tota l'operació dins d'una única transacció de manera natural, cosa que el patró *Repository* per si sol no facilita.

A més, la transacció s'obre a la capa de controlador i el servei s'executa íntegrament sota aquest context, cosa que evita les transaccions niuades: un servei pot cridar-ne un altre sense obrir cap transacció pròpia, ja que totes comparteixen la del controlador.

7.2.3 PostgreSQL

La base de dades del sistema és PostgreSQL [31]. La primera decisió, prèvia al producte concret, és el model de dades: es va optar per una base de dades relacional perquè les dades ho són (usuaris que tenen documents, documents amb destinataris, destinataris amb signatures i esdeveniments associats), i el model relacional garanteix per disseny la coherència d'aquestes relacions. Una base de dades no-relacional aportaria una flexibilitat d'esquema que el projecte no necessita; en canvi, sí que necessita les garanties transaccionals fortes del model relacional (vegeu la Secció 7.2.2).

Dins del model relacional, PostgreSQL es va triar perquè és programari lliure i gratuït, és un dels motors més madurs i robustos pel que fa al comportament transaccional (compleix les propietats ACID) i té un fort suport tant a TypeORM com a l'ecosistema Node, del qual és l'estàndard *de facto*. Alternatives com MySQL o MariaDB haurien estat igualment viables i cap requisit les descartava.

7.2.4 Criptografia: Ed25519

Per a la signatura digital es van avaluar tres esquemes de clau pública: RSA, ECDSA i Ed25519 [23]. Es va seleccionar Ed25519 principalment per una propietat: el determinisme.

ECDSA genera cada signatura amb l'ajuda d'un valor aleatori. Si aquest valor és feble o es reutilitza, es pot arribar a deduir la clau privada, un problema que ha causat compromisos de seguretat reals i coneguts (el cas més famós és el de Sony i la PlayStation 3, on la reutilització d'aquest valor va permetre recuperar la clau privada amb què se signava el programari de la consola; l'anàlisi del cas, però, queda fora de l'abast d'aquesta memòria). Ed25519, en canvi, deriva aquest valor de manera determinista a partir del propi missatge i de la clau, de manera que no depèn de cap generador d'aleatorietat.

Pel que fa a RSA, que no presenta aquest problema concret, es va descartar per una qüestió pràctica: per assolir un nivell de seguretat equivalent necessita claus i signatures molt més grans (de l'ordre de milers de bits, davant de les poques desenes de bytes d'Ed25519) i amb operacions més costoses. En un sistema que emmagatzema una signatura per cada acte de signatura, la compacitat d'Ed25519 és un avantatge directe. A tot això s'hi afegeix que és un estàndard modern i àmpliament adoptat en protocols actuals. La implementació es fa mitjançant el mòdul criptogràfic natiu de Node.js, sense dependències externes. L'eIDAS és neutre pel que fa a l'algorisme: la tria d'Ed25519 no altera el nivell de signatura (SES), però sí que reforça la qualitat tècnica i probatòria de l'evidència.

7.3 Pila del *frontend*

7.3.1 Next.js i React

La tria de Next.js [26] (que estén la biblioteca React [27]) prové, en part, de la decisió de coherència de llenguatge ja exposada: TypeScript a tot el projecte. A més a més, hi ha dos motius importants que acaben de justificar l'elecció.

El primer és el model d'encaminament i renderització. Amb l'*App Router* de Next.js, les rutes es defineixen per l'estructura de fitxers i cada pàgina es resol i es renderitza inicialment al servidor (SSR), en lloc de deixar tot l'encaminament en mans del navegador com faria una SPA clàssica amb un encaminador de client. Això millora el temps de càrrega inicial (en línia amb el requisit de rendiment del Capítol 6) i fa que qualsevol URL de la plataforma funcioni directament, sense haver de carregar primer tota l'aplicació.

El segon és l'ecosistema: Next.js aporta de sèrie l'estructura i les optimitzacions descrites al Capítol 5 (empaquetat, càrrega diferida, optimització d'imatges), amb una comunitat i un manteniment molt actius. Per a l'abast d'aquest projecte, qualsevol *framework* de *frontend* modern hauria estat funcionalment suficient; la combinació de coherència de llenguatge, encaminament, SSR i ecosistema és el que va decantar la tria.

7.3.2 MUI i Tailwind

Per a la interfície es va descartar construir un sistema de components propi: fer-ho a mà suposa un cost enorme (camps de formulari, diàlegs, taules, estats de focus i error) i, sobretot, fa molt difícil assolir un bon nivell d'accessibilitat. Es va optar per MUI [32], una biblioteca de components madura que implementa les pautes WAI-ARIA [20] de sèrie, en línia amb el compromís d'accessibilitat declarat a la viabilitat ètica (Capítol 2).

La capa visual combina, però, dues eines que poc sovint s'utilitzen alhora: MUI i Tailwind [33]. El seu ús conjunt en aquest projecte ve d'una separació de responsabilitats premeditada. Tailwind s'utilitza exclusivament com a capa de *tokens* de disseny: defineix la paleta de colors i les variables per als modes clar i fosc en un únic punt. MUI s'encarrega de tot el sistema de components de la interfície, emprant aquestes variables. D'aquesta manera es defineix un sistema de disseny amb Tailwind i un únic sistema de components amb MUI, sense que les dues eines competeixin per l'estil dels mateixos elements.

7.3.3 Client d'API propi

Per a la comunicació amb l'API es va descartar l'ús de biblioteques habituals com `axios` o gestors d'estat de servidor com `React Query`, i es va optar per un client propi construït sobre l'API nativa `fetch`. El motiu principal és la desproporció entre el que aporten i el que es necessita: `axios` incorpora tot un ventall de funcionalitats (interceptors, transformacions automàtiques, cancel·lació de peticions, seguiment del progrés de pujada, adaptadors per a diferents entorns) de les quals el projecte només usaria una fracció mínima, i `React Query` hi afegeix una capa sencera de memòria cau i revalidació d'estat de servidor que aquest abast no requereix. Un client propi sobre `fetch` cobreix el necessari, sense dependències, i manté el control total sobre el comportament de les peticions.

Aquesta decisió va tenir una conseqüència tècnica curiosa. En implementar el refresc silencios del *token* de sessió, va aparèixer un problema de concurrència: si diverses peticions caducaven alhora, totes intentaven refrescar el *token* simultàniament i, com que el *backend* invalida el *token* antic en emetre'n un de nou, totes les peticions menys la primera fallaven i provocaven un tancament de sessió no desitjat. La solució va ser un mecanisme de bloqueig (*lock*) que garanteix que només s'executi un refresc alhora, mentre la resta de peticions n'esperen el resultat i es reintenten amb el *token* nou. Aquest comportament està cobert per proves automàtiques.

7.4 Infraestructura i desplegament

7.4.1 VPS enfront de plataformes gestionades

La decisió entre un VPS i una plataforma gestionada (PaaS) ja s'ha argumentat a la viabilitat tècnica (Capítol 2) i es resumeix aquí: les solucions *click-to-deploy* com Vercel o Netlify amaguen completament la infraestructura al desenvolupador, i el control i l'aprenentatge d'aquesta capa eren objectius explícits del projecte. Un VPS sobre KVM ofereix el control total que es buscava, a un cost molt inferior al de les grans plataformes *cloud*.

7.4.2 Dokploy

Dokploy [12] es va triar com a punt intermedi idoni: una plataforma autoallotjada sobre Docker que aporta les capacitats d'operació professional (desplegaments amb *rollback*, *logs*, còpies de seguretat, certificats TLS automàtics mitjançant Traefik [30] i composició de serveis) sense la complexitat de les grans solucions. Permet, així, entendre i controlar de prop tot el cicle de desplegament, que és precisament el que es buscava. Dins de l'espai de les plataformes autoallotjades existeixen alternatives comparables (com Coolify o CapRover); no se'n va fer una comparativa formal, ja que la plataforma era un mitjà i no un fi: Dokploy va cobrir totes les necessitats identificades des de les primeres proves i no va donar cap motiu per reconsiderar la tria.

7.4.3 Emmagatzematge: Garage

Com a servei d'emmagatzematge d'objectes es va desplegar Garage [34], una implementació autoallotjada i de codi obert del protocol S3 [35], emprada com a destinació de les còpies de seguretat de la plataforma. La tria segueix la mateixa lògica que la

resta de la infraestructura: control total, aprenentatge del protocol estàndard (S3) i cost zero, davant de l'alternativa d'un servei d'emmagatzematge al núvol de tercers.

7.5 Eines de desenvolupament i qualitat

El control de versions és Git, amb GitHub com a plataforma: *Pull Requests*, protecció de branques i GitHub Actions [10] per als *pipelines*. El flux de treball (GitFlow) i el cicle de CI/CD ja s'han descrit al Capítol 3. Docker [11] s'emptra a totes les fases: entorns locals reproduïbles, execució de proves i desplegament a producció.

Per a l'anàlisi estàtica de seguretat (SAST) es va avaluar inicialment SonarCloud, que es va descartar pel cost de la versió al núvol i per la complexitat de configurar-ne l'alternativa autoallotjada (SonarQube). Es va optar per Sengrep [36]: gratuït, ràpid i directament integrable com un pas més del *pipeline* de CI, on s'executa a tots dos repositoris.

Pel que fa a les proves, al *backend* s'utilitza Jest, l'entorn de proves que NestJS integra de sèrie; al *frontend*, Vitest, més ràpid i amb suport natiu del TypeScript modern, juntament amb Playwright per a les proves E2E. El detall de l'estratègia de proves es tracta al Capítol 9.

Capítol 8

Anàlisi i disseny del sistema

L'anàlisi estudia amb detall com s'han d'interpretar els requisits del sistema (Capítol 6); el disseny en proposa una solució concreta. El capítol s'estructura en dues parts: l'anàlisi (rols, dades, casos d'ús i model conceptual) i el disseny (base de dades, interfície i patrons). Les tecnologies ja justificades al Capítol 7 no es repeteixen aquí.

8.1 Anàlisi

8.1.1 Tipus d'usuari i rols

Els actors del sistema ja s'han introduït al Capítol 6: l'usuari registrat, el destinatari extern i el propi sistema. Des del punt de vista del rol funcional, hi destaquen dos aspectes.

L'usuari registrat és un únic rol homogeni: un cop autenticat, té les mateixes capacitats sobre els seus propis recursos que qualsevol altre usuari registrat, sense cap panell d'administració ni operació reservada a un rol privilegiat. El sistema preveu internament un camp de rol al context de petició, però no arriba a assignar-hi cap valor: el projecte no implementa control d'accés basat en rols (RBAC). Només amb una orientació B2B (Capítol 3) hauria calgut algun rol organitzatiu, com un administrador d'empresa; no va ser el cas, i queda fora de l'abast d'aquest projecte.

El destinatari extern, en canvi, no té compte: la seva identitat es limita a una adreça de correu i, opcionalment, un nom, i la sessió que se li concedeix és temporal i lligada a un únic enllaç públic. Aquesta diferència entre usuari amb compte i destinatari sense compte es nota a tot el disseny: des de l'autenticació (JWT davant de sessió pública) fins al model de dades (Secció 8.1.4).

8.1.2 Anàlisi dels requisits

No es repeteix aquí la taula de requisits (Capítol 6); aquest apartat n'agrupa les implicacions de dades i de procés pels mateixos blocs funcionals que s'utilitzen al model conceptual (Secció 8.1.4), de manera que es cobreixen tots els requisits i no només els que generen més dades.

Els requisits d'identitat (RF1, gestió d'usuaris; RF2, contactes) es tradueixen, en principi, en un model d'usuari i una llista de contactes independent. El matís important és a RF1: esborrar un compte no es limita a anonimitzar-ne les dades personals, sinó

que desencadena dos efectes en cascada. Si l'usuari eliminat és emissor, el document passa a cancel·lat i se n'avisava els destinataris pendents; si n'és destinatari, la seva línia de signatura passa a caducada, se'n registra l'esdeveniment i se n'avisava l'emissor. Aquest requisit, doncs, no afecta només la taula d'usuaris, sinó que es propaga al model de documents i destinataris.

Els requisits de gestió i enviament de documents (RF3, RF4) impliquen versionar-los (un document pot tenir diverses versions encadenades) i seguir l'estat de cada destinatari per separat, no del document en conjunt: dos destinataris del mateix document poden estar en estats diferents alhora. El bloqueig d'accés (RF10) hi afegeix que la protecció d'un document no és una simple contrasenya guardada al mateix document, sinó una relació 1-N (un document pot tenir més d'un bloqueig) i, per a cada bloqueig, un registre independent de quins destinataris ja l'han resolt. L'accés de destinataris externs (RF11) hi suma un tercer element: una sessió temporal pròpia, independent de la de l'usuari registrat.

El nucli de dades del sistema és el bloc de signatura (RF5–RF9). Signar un document no és només marcar-lo com a signat: cal conservar tota l'evidència associada a aquell acte concret (l'artefacte criptogràfic, l'empremta i el *payload* exacte que es va signar, i la referència a la verificació d'identitat per OTP que el va precedir), i fer-ho per a cada destinatari per separat, ja que cadascun signa de manera independent. El requisit de verificació independent (RF8) exigeix, a més, que cada artefacte sigui autocontingut: ha d'incloure la seva pròpia clau pública, no només confiar en una clau global del sistema, de manera que la verificació d'un artefacte antic seguiria sent vàlida fins i tot si en el futur s'arribés a canviar la clau de signatura (el propi model ja preveu un camp de versió de clau per a aquest cas, tot i que avui totes les signatures en fan servir la mateixa). Finalment, l'historial (RF9) implica un tipus de dada diferent de l'evidència de signatura: un registre d'esdeveniments *append-only* que no depèn de l'estat actual dels destinataris per ser fiable.

Els requisits restants (RF12, internacionalització; RF13, retenció i purga) no afegeixen entitats noves al model de dades: el primer és una qüestió de contingut (fitxers de missatges), i el segon només llegeix les marques de temps ja existents per decidir què esborrar.

8.1.3 Casos d'ús

Un cop analitzats els requisits, la Figura 8.1 en resumeix els casos d'ús principals i els actors que hi intervenen. Les fitxes detallades dels cinc casos més rellevants (enviar un document, resoldre'l, revocar-ne la signatura, verificar-la de manera independent i accedir com a destinatari extern) es recullen a l'Apèndix A.

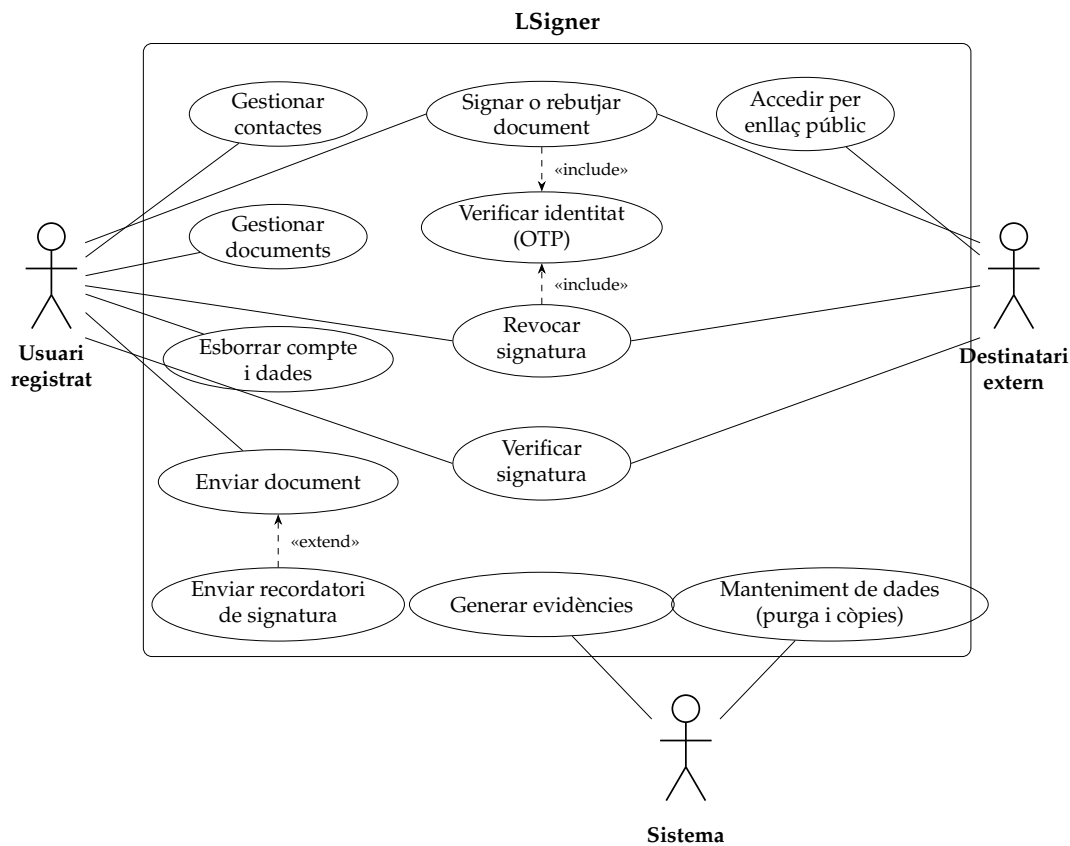


FIGURA 8.1: Diagrama de casos d'ús del sistema.

8.1.4 Model conceptual de dades

El model de dades es representa amb el diagrama de classes de la Figura 8.2: onze entitats, agrupades en quatre blocs. El bloc d'identitat (*User*, *Contact*, *RefreshToken*) gestiona els comptes i les seves relacions de confiança. El bloc de documents (*Document*, *DocumentLock*, *DocumentLockResolution*) gestiona el cicle de vida del fitxer i les seves proteccions d'accés. El bloc de signatura (*DocumentRecipient*, *DocumentSignedArtifact*, *DocumentSigningEvent*) gestiona qui ha de signar, l'evidència que en resulta i l'historial immutable d'accions. Finalment, el bloc d'accés temporal (*PublicLinkSession*, *OtpChallenge*) dona suport a l'autenticació de destinataris externs i a la verificació d'identitat per OTP.

DocumentRecipient és el centre de tot: hi conflueixen el document que cal signar, l'usuari registrat opcional que el signa (*user_id* és nul quan el destinatari és extern), l'evidència de la signatura i l'historial d'accions. *OtpChallenge*, en canvi, apareix aïllat a la figura: per disseny, és genèric i no té cap relació d'ORM cap a cap altra entitat, ni tan sols cap a l'usuari que ha de verificar-se. En lloc de claus foranes, guarda uns identificadors solts (*user_id* i *resource_type/resource_id*) que, avui, sempre apunten a un usuari i a un document, respectivament. Sobre aquests mateixos camps hi ha un índex compost, però amb una finalitat diferent de la que podria semblar: no accelera la verificació del repte (que sempre es fa per *id*), sinó la cerca d'un repte actiu ja existent en crear-ne un de nou, per evitar-ne la duplicació.

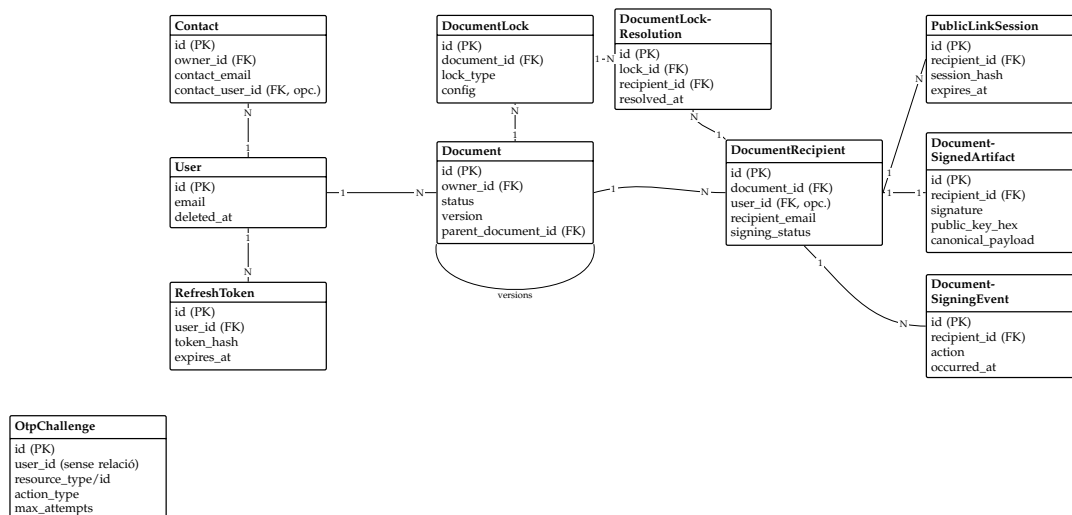


FIGURA 8.2: Model conceptual de dades del sistema: entitats i relacions principals (claus i camps no essencials omesos). *OtpChallenge* apareix aïllat perquè no té cap relació d'ORM.

8.2 Disseny

8.2.1 Disseny de la base de dades

El sistema utilitza el model relacional justificat al Capítol 7 (PostgreSQL amb TypeORM). Aquest apartat detalla, taula per taula, els camps, les claus i els tipus de les quatre entitats centrals del flux de signatura. El detall de la resta de taules (usuaris, contactes, *tokens* de sessió, sessions d'enllaç públic, bloquejos i les seves resolucions, i reptes OTP) es recull a l'Apèndix B per no allargar aquest capítol.

TAULA 8.1: Disseny de la taula documents.

Camp	Tipus	Clau	Descripció
id	uuid	PK	Identificador del document.
owner_id	uuid	FK <i>users</i>	Usuari propietari (<i>emissor</i>); esborrat en cascada.
title	varchar(255)		Títol del document.
description	text		Descripció opcional.
file	bytea		Contingut binari; no es carrega per defecte.
file_hash	char(64)		Empremta SHA-256 del contingut, per integritat.
original_filename	varchar(255)		Nom del fitxer original.
mime_type	varchar(100)		Tipus MIME del fitxer.
file_size	bigint		Mida en bytes.
status	enum		DRAFT / SENT / SUPERSEDED / VOIDED / CANCELLED / DELETED.
version	int		Número de versió (comença a 1).

Camp	Tipus	Clau	Descripció
parent_document_id	uuid	FK documents	Versió anterior de la cadena; nul a la primera.
created_at/updated_at	timestampz		Marques de temps de creació i modificació.

TAULA 8.2: Disseny de la taula document_recipients.

Camp	Tipus	Clau	Descripció
id	uuid	PK	Identificador de la línia de destinatari.
document_id	uuid	FK documents	Document al qual pertany; esborrat en cascada.
user_id	uuid	FK users	Nul si el destinatari és extern (sense compte).
recipient_email	varchar(255)		Adreça del destinatari.
recipient_name	varchar(200)		Nom opcional del destinatari.
public_link_id	varchar(64)		Identificador de l'enllaç públic (destinatari externs).
status	enum		Estat d'entrega: PENDING / UPDATED.
sent_at	timestampz		Data d'enviament.
first/last_accessed_at	timestampz		Primer i darrer accés a l'enllaç.
signing_status	enum		PENDING / SIGNED / REJECTED / REVOKED / EXPIRED.
signed_at	timestampz		Data de signatura, si escau.
created_at/updated_at	timestampz		Marques de temps.

Restricció d'unicitat: (document_id, recipient_email); un mateix correu no pot rebre el mateix document dues vegades.

TAULA 8.3: Disseny de la taula document_signed_artifacts.

Camp	Tipus	Clau	Descripció
id	uuid	PK	Identificador de l'artefacte.
document_id	uuid	FK documents	Document signat.
recipient_id	uuid	FK document_recipients	Destinatari que signa (relació 1-1).
file	bytea		Document signat resultant.
file_hash	char(64)		Empremta SHA-256 del document signat.
signature	text		Signatura Ed25519 en base64.

Camp	Tipus	Clau	Descripció
signature_algorithm	varchar(50)		Sempre Ed25519.
key_fingerprint	char(64)		Empremta SHA-256 de la clau pública.
key_version	int		Versió de la clau emprada per signar.
previous_artifact_id	uuid		Artefacte anterior de la mateixa cadena de custòdia.
public_key_hex	char(64)		Clau pública en hexadecimal, per verificació autocontinguda.
canonical_payload	text		<i>Payload</i> JSON canònic exacte que es va signar.
evidence	jsonb		Evidència addicional estructurada.
signed_at	timestampz		Data i hora de la signatura.
created_at	timestampz		Marca de temps de creació del registre.

Restricció d'unicitat: (document_id, recipient_id); com a màxim un artefacte per destinatari i document.

TAULA 8.4: Disseny de la taula document_signing_events.

Camp	Tipus	Clau	Descripció
id	uuid	PK	Identificador de l'esdeveniment.
document_id	uuid	FK documents	Document al qual fa referència.
recipient_id	uuid	FK document_recipients	Destinatari que origina l'esdeveniment.
action	enum		ACCESS_OPENED / SIGNED / REJECTED / REVOKED / RECIPIENT_ACCOUNT_DELETED.
metadata	jsonb		Dades addicionals de context de l'acció.
occurred_at	timestampz		Moment exacte de l'esdeveniment.
created_at	timestampz		Marca de temps de creació del registre.

Aquesta taula no s'actualitza mai un cop escrita: és el registre *append-only* que sustenta l'historial (RF9) i l'evidència de no-repudi (RNF4).

8.2.2 Disseny de les interfícies d'usuari

La interfície es va dissenyar sota una filosofia de navegació senzilla i consistent, en línia amb el requisit d'usabilitat (RNF7). L'estructura és la típica d'una aplicació de gestió (barra lateral de navegació, barra superior i contingut principal), en comptes d'una navegació basada en pestanyes o en un únic flux lineal: la barra lateral dona accés directe a les seccions principals (documents rebuts, documents enviats, contactes, historial, configuració) sense necessitat de navegar per nivells.

Les accions que trenquen el flux principal (confirmar una acció, mostrar el detall d'un element) es resolen amb finestres modals en comptes de pàgines noves, per evitar la pèrdua de context: l'usuari no perd de vista la llista sobre la qual estava treballant. L'excepció és l'enviament d'un document, que per la seva complexitat (pujada, metadades, destinataris, confirmació) es resol amb un assistent de diversos passos (*wizard*) dins d'un mateix flux modal, en comptes d'un únic formulari llarg: cada pas té la seva pròpia validació i l'usuari sempre sap en quin punt del procés es troba.

La resta d'aspectes visuals (paleta de colors, mode fosc i clar, sistema de components) ja s'han justificat al Capítol 7.

8.2.3 Patrons de disseny aplicats

Ja s'han descrit al llarg d'aquesta memòria dos patrons rellevants: l'organització per capes a l'estil MVC (Capítol 5) i l'obertura de la transacció al controlador amb l'`EntityManager` (Capítol 7). Aquest apartat en recull cinc més, aplicats de manera conscient.

Guard global amb excepció explícita (*secure by default*). Per defecte, cap ruta de l'API és accessible sense autenticació: un *guard* global exigeix un JWT vàlid a totes les peticions. Les rutes que han de ser públiques (registre, inici de sessió i verificació independent de signatures) es marquen explícitament amb un decorador. D'aquesta manera, oblidar-se de marcar una ruta nova com a pública és l'error habitual, en comptes que oblidar-se de protegir-la ho sigui: el comportament per defecte ja és el segur.

Registre i estratègia per als bloquejos de documents. El sistema de bloqueig de documents (RF10) es va dissenyar per admetre més d'un tipus de protecció, tot i que de moment només s'ha implementat la contrasenya. Cada tipus de bloqueig implementa una mateixa interfície (aplicar i verificar), i un registre central en manté la correspondència entre tipus i implementació. Afegir un nou tipus de bloqueig en el futur només requeriria implementar aquesta interfície i registrar-la, sense tocar la lògica de documents o destinataris que ja en fa ús.

Historial d'esdeveniments immutable. El sistema manté un camp d'estat mutable a cada destinatari per a consultes ràpides (`signing_status`), però cada canvi rellevant es complementa amb un esdeveniment *append-only* que en deixa constància permanent. Així, l'historial complet d'un document és sempre reconstruïble a partir dels esdeveniments, cosa que reforça la traçabilitat exigida pel requisit de no-repudi (RNF4).

Filtre d'excepcions global. Totes les excepcions de l'API passen per un únic filtre que les normalitza a una mateixa forma JSON. Això evita que cada controlador hagi de formatar els seus propis errors, i garanteix que el *frontend* sempre rep errors amb la mateixa forma, vingui d'on vingui.

Mapper entre entitats i DTO. Les entitats de TypeORM no s'exposen mai directament a l'API: una capa de funcions de mapatge en deriva les formes de resposta que necessita cada *endpoint*. Això desacobla el model de persistència del contracte de l'API: canviar l'estructura interna d'una entitat no obliga a canviar la forma en què el *frontend* la rep, sempre que el *mapper* s'actualitzi en conseqüència.

Capítol 9

Implementació i proves

Els capítols anteriors ja han explicat què és cada component del sistema (Capítol 5), per què es va triar cada tecnologia (Capítol 7) i com s'estructuren les dades i els casos d'ús (Capítol 8). Aquest capítol detalla el que falta: com s'ha implementat realment el codi, quins problemes concrets han sorgit durant el desenvolupament i com s'han resolt, i com es verifica que tot funciona.

9.1 Arquitectura general del sistema

La Figura 9.1 resumeix els components principals del sistema i com es relacionen.

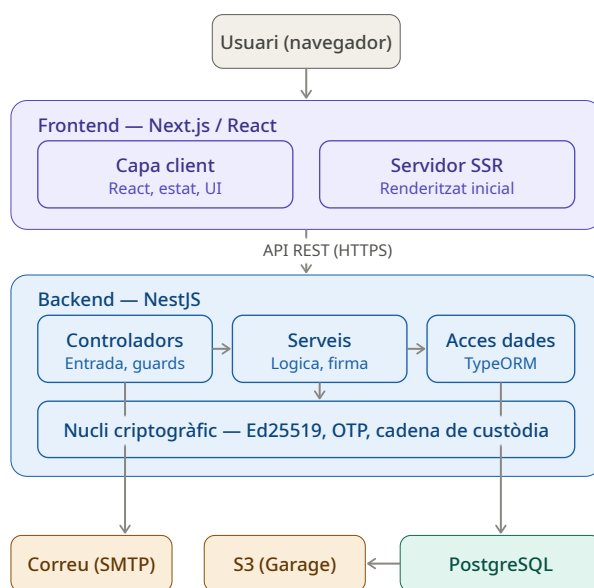


FIGURA 9.1: Arquitectura general del sistema: components principals i les seves relacions.

El navegador de l'usuari parla amb el *frontend* (Next.js), que té dues capes: una de client (React, estat, interfície) i una de servidor per a la renderització inicial (SSR). El *frontend* es comunica amb el *backend* exclusivament per API REST sobre HTTPS. Dins del *backend*, la petició travessa les tres capes ja conegudes (controladors, serveis,

accés a dades) i, quan cal, el nucli criptogràfic (Ed25519, OTP, cadena de custòdia) que es detalla a la Secció 9.2.1.

Un detall no segueix el patró habitual de capes: el correu (els codis OTP i les notificacions de documents) s'envia directament des dels controladors, no des dels serveis; per exemple, `otp.controller.ts` injecta i crida `EmailService` ell mateix. Un controlador propi per enviar correus no té sentit: els correus són conseqüència d'acions com signar un document, i és el mateix mòdul qui crida el servei de correu.

La base de dades PostgreSQL és el punt d'accés a dades del *backend*. La fletxa cap a S3 (Garage) surt directament de PostgreSQL perquè és un procés d'infraestructura extern a l'aplicació: un *cron* en l'àmbit del VPS que fa còpies de seguretat de la base de dades, no una funcionalitat exposada per l'API.

9.2 Desenvolupament dels mòduls principals

Aquesta secció recull els mòduls i decisions d'implementació més rellevants, amb els problemes concrets que van sorgir i com es van resoldre.

9.2.1 El nucli de signatura: payload canònic i Ed25519

Signar un document no es redueix a cridar una funció de xifratge: cal construir primer un *payload* que representi, sense ambigüitat, què s'està signant. La funció `buildSignatureEvidence` (`document-signing.service.ts`) en construeix un a partir de camps rellevants de l'acte de signatura (document, destinatari, estat, moment) i el converteix a una cadena JSON amb les claus ordenades alfabèticament abans de signar-lo:

LISTING 9.1: Construcció del *payload* canònic i firma Ed25519 (`document-signing.service.ts`)

```
1 const canonicalPayload = {
2   actor_id: recipient.id,
3   actor_type: 'RECIPIENT',
4   document_id: document.id,
5   file_hash: document.file_hash,
6   key_version: keyVersion,
7   previous_artifact_id: previousArtifactId,
8   recipient_id: recipient.id,
9   state,
10  timestamp: now.toISOString(),
11 };
12
13 // Deterministic JSON with sorted keys
14 const canonical = JSON.stringify(
15   canonicalPayload,
16   Object.keys(canonicalPayload).sort(),
17 );
18
19 const message = Buffer.from(canonical, 'utf8');
20 const signatureBuffer = crypto.sign(null, message, privateKey.privateKey);
21 const signature = signatureBuffer.toString('base64');
```

L'ordenació de claus és necessària perquè `JSON.stringify` no garanteix cap ordre per defecte: sense fixar-lo, el mateix esdeveniment es podria serialitzar de maneres diferents i generar firmes no reproduïbles bit a bit.

Cada artefacte de signatura es guarda amb la seva pròpia clau pública (`public_key_hex`) i l'empremta corresponent (`key_fingerprint`), derivades una única vegada en arrencar el servei a partir de les claus Ed25519 configurades per entorn:

LISTING 9.2: Derivació de l'empremta i la clau pública en cru (`signing.config.ts`)

```
1 const fingerprint = crypto
2   .createHash('sha256')
3   .update(publicKeyDer)
4   .digest('hex');
5
6 // Raw 32-byte Ed25519 public key as hex (for external verification tools)
7 const publicKeyRaw = publicKey.export({ type: 'spki', format: 'der' });
8 // Ed25519 SPKI DER = 30 2A 30 05 06 03 2B 65 70 03 21 00 + 32 raw bytes
9 // The raw key starts at byte 12
10 const publicKeyHex = publicKeyRaw.subarray(12).toString('hex');
```

Aquest desplaçament de 12 bytes elimina la capçalera fixa que Node afegeix en exportar la clau en format SPKI DER, deixant només els 32 bytes de la clau en cru que qualsevol eina externa de verificació Ed25519 espera rebre.

Un problema real d'aquesta part va sorgir amb la concurrència: si dues peticions de signatura arriben gairebé alhora (per exemple, un doble clic), totes dues poden arribar a comprovar l'estat del destinatari abans que cap de les dues l'hagi actualitzat, i totes dues el troben «pendent de signar». Sense cap altra mesura, això acabaria creant dos artefactes de signatura per al mateix destinatari. La protecció real no és aquesta comprovació prèvia, sinó la restricció d'unicitat de la base de dades sobre (`document_id`, `recipient_id`) (Secció 8.2.1): la primera petició escriu sense problemes, i la segona hi xoca (codi d'error 23505 de PostgreSQL), cosa que el codi converteix en un missatge d'error clar en comptes d'un error intern:

LISTING 9.3: La restricció d'unicitat de la BD com a última defensa contra la doble signatura

```
1 try {
2   savedArtifact = await entityManager.save(artifact);
3 } catch (err) {
4   if (
5     err instanceof QueryFailedError &&
6     (err as QueryFailedError & { code: string }).code === '23505'
7   ) {
8     throw new ConflictException('This document has already been signed');
9   }
10  throw err;
11 }
```

9.2.2 Verificació d'identitat (OTP) i el seu vincle amb la firma

El model conceptual (Secció 8.1.4) ja explica per què cal un OTP abans de signar; aquí es detalla com queda lligat, en el codi, a l'artefacte que en resulta. Quan el destinatari introdueix el codi rebut per correu, el controlador d'OTP obre una única transacció que engloba tant la verificació del codi com l'execució de la signatura: si la signatura falla per qualsevol motiu, tota la transacció fa *rollback* i l'OTP no es dona per consumit, de manera que el destinatari el pot tornar a fer servir.

LISTING 9.4: Verificació d'OTP i signatura dins de la mateixa transacció (`otp.controller.ts`)

```

1 return this.entityManager.transaction(
2   async (transactionalEntityManager) => {
3     const challenge = await this.otpService.verifyChallenge(
4       challengeId,
5       dto,
6       transactionalEntityManager,
7     );
8     // ...
9     return this.documentSigningService.executeOtpActionByUserId(
10      challenge,
11      user.sub,
12      transactionalEntityManager,
13    );
14  },
15 );

```

El vincle entre l'OTP i la firma no és només temporal: l'identificador del *challenge* viatja explícitament com a *verification_reference*, tant dins de l'evidence de l'artefacte com dins del *metadata* de l'esdeveniment de signatura corresponent. Això permet respondre, per a qualsevol firma concreta, exactament quina verificació d'identitat la va autoritzar, sense haver de confiar només en l'ordre temporal dels registres.

9.2.3 Gestió de sessió al *frontend*: refresc concurrent

El *frontend* renova l'*access token* de manera silenciosa quan caduca (15 minuts de durada). Aquest comportament va destapar un error habitual: en carregar una pantalla amb diverses crides en paral·lel, totes rebien el 401 gairebé alhora i cadascuna disparava el seu propi refresc. Com que el *refresh token* es rota a cada ús (Capítol 8), la segona renovació trobava el *token* ja invalidat per la primera i fallava.

La solució, a *client.ts*, comparteix una única promesa entre totes les peticions concurrents en comptes de deixar que cadascuna en dispari una de pròpia:

LISTING 9.5: *withRefreshLock*: un sol refresc actiu per a totes les peticions concurrents (*client.ts*)

```

1 let refreshPromise: Promise<boolean> | null = null;
2
3 async function withRefreshLock(
4   refreshFn: () => Promise<boolean>,
5 ): Promise<boolean> {
6   if (refreshPromise) return refreshPromise;
7   refreshPromise = refreshFn().finally(() => {
8     refreshPromise = null;
9   });
10  return refreshPromise;
11 }

```

La primera petició que troba un 401 inicia el refresc i en guarda la promesa; qualsevol altra que arribi mentre encara està en curs simplement s'hi subscriu, sense iniciar-ne un altre.

La gestió de sessió dels destinataris externs (que no fan servir JWT) segueix un mecanisme relacionat però propi, detallat a l'Apèndix C.

9.2.4 Docker multietapa

Tant el *backend* com el *frontend* es construeixen amb imatges Docker multietapa (`node:20-alpine`): una etapa instal·la totes les dependències i compila, i una etapa final només hi copia el resultat i les dependències de producció, sense eines de *build* ni codi font. Es fa així per mantenir la imatge que acaba corrent en producció la més petita i neta possible: la imatge de producció del *backend*, mesurada directament sobre la construcció actual del repositori, pesa 525 MB; l'etapa intermèdia, que instal·la totes les dependències incloent-hi les de desenvolupament, és una mica més gran, ja que hi entren totes les eines de compilació i de proves que la imatge final no necessita.

Un cas particular d'aquesta estratègia, la imatge de desenvolupament del *frontend* amb Chromium per a Playwright, es detalla a l'Apèndix C.

9.3 Integració

La integració entre totes les peces s'entén molt millor amb un exemple visual. La Figura 9.2 en mostra un cas central del sistema: firmar un document com a destinatari.

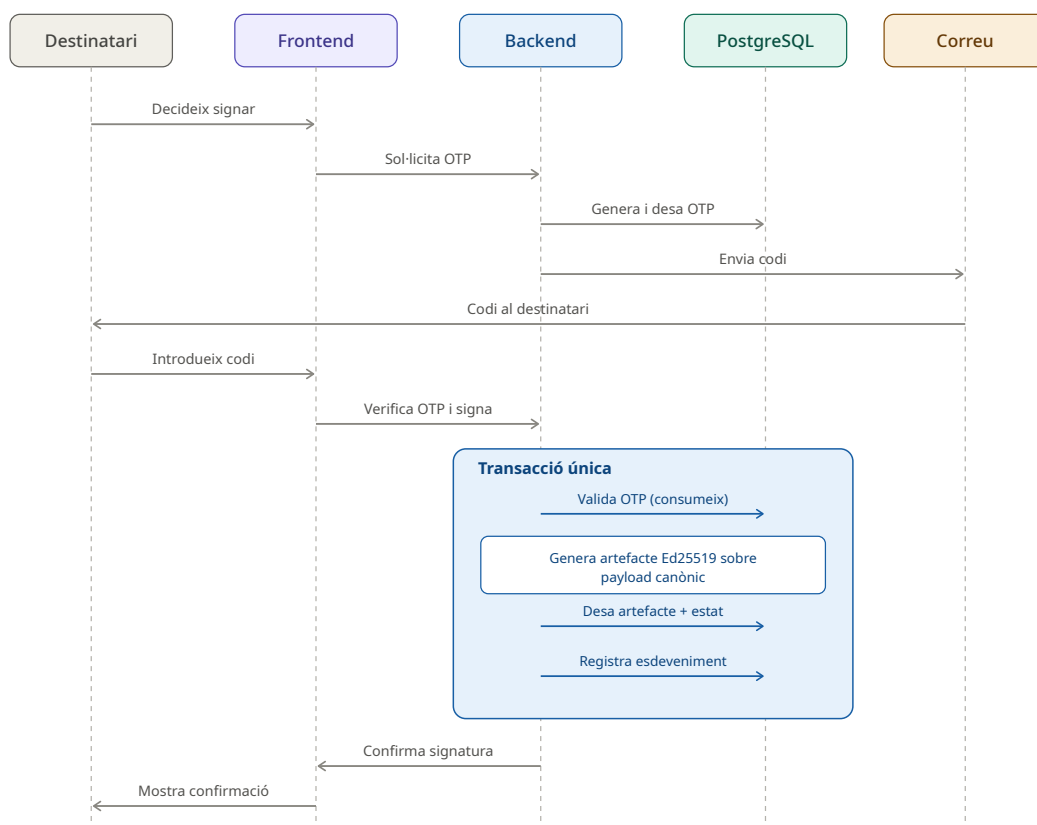


FIGURA 9.2: Seqüència de firma d'un document: sol·licitud i verificació d'OTP, i execució de la firma dins d'una única transacció.

El destinatari decideix signar i el *frontend* sol·licita un OTP al *backend*, que el genera, el desa i l'envia per correu; el destinatari el rep i el torna a introduir al *frontend*, que el reenvia al *backend* per verificar-lo. A partir d'aquí, tot passa dins d'una única

transacció (Secció 9.2.2): es valida i consumeix l'OTP, es genera l'artefacte Ed25519 sobre el *payload* canònic, es desa l'artefacte i s'actualitza l'estat del destinatari, i es registra l'esdeveniment corresponent. Si tot això es compleix, el *backend* confirma la signatura i el *frontend* en mostra la confirmació a l'usuari.

Pel que fa al desplegament, la Figura 9.3 mostra com s'organitzen els mateixos components dins de la infraestructura real (Capítol 7).

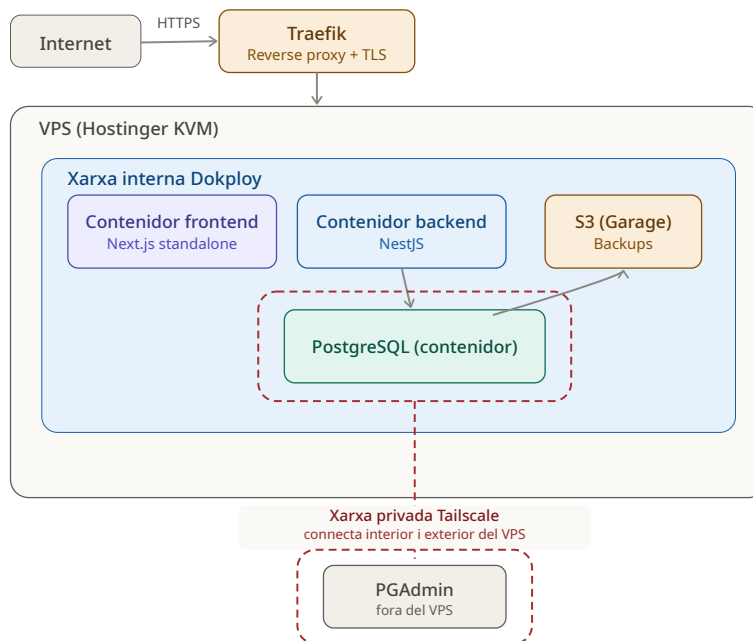


FIGURA 9.3: Topologia de desplegament: Traefik, la xarxa interna de Dokploy i l'accés restringit per Tailscale.

Totes les peticions externes entren per Traefik, que en gestiona el TLS i les encamina cap al contenidor corresponent dins de la xarxa interna de Dokploy (*frontend*, *backend*, PostgreSQL i l'emmagatzematge S3 de les còpies de seguretat). Al diagrama, Traefik es dibuixa fora del requadre de Dokploy per remarcar que HTTPS és l'únic port de servei obert cap al VPS; a la pràctica, Traefik s'executa com un contenidor més gestionat pel mateix Dokploy, que és qui en defineix les rutes permeses. PostgreSQL, en concret, té una via d'accés addicional que no passa per Traefik ni per l'API: la xarxa privada de Tailscale, que l'envolta per dins del VPS i s'estén cap enfora fins a PGAdmin, l'eina d'administració de la base de dades que s'executa fora del VPS. És, doncs, l'única manera d'arribar-hi directament des de fora sense passar per l'aplicació.

9.4 Proves i validació

9.4.1 Estratègia de testing

El *backend* i el *frontend* segueixen estratègies de proves diferents, adaptades a cada capa.

Al *backend*, les proves unitàries (Jest) simulen l'*EntityManager* i les cadenes de *QueryBuilder* (`mockReturnThis()`): no hi ha cap base de dades real, sinó objectes que imiten l'accés i en retornen les dades que la prova indica, com si vinguessin d'una consulta real. Són ràpides i deterministes, però no detecten problemes reals d'integració (una migració mal aplicada, una restricció de BD que no es comporta com s'esperava). Per això hi ha, a més, una suite de proves *end-to-end* (Supertest) que aixeca l'aplicació sencera contra una base de dades PostgreSQL real, recreada abans de cada execució (`prepare-e2e-db.ts`): aquestes són les que haurien de detectar errors de comportament real, com el cas de doble signatura descrit a la Secció 9.2.1.

Al *frontend*, les proves unitàries (Vitest i Testing Library) proven cada funcionalitat fent servir una API falsa, amb respostes simulades preparades per a cada cas (`vi.mock`). Les proves *end-to-end* (Playwright), en canvi, llancen un navegador real amb el *frontend* real: la diferència és que les peticions cap al *backend* queden interceptades (`page.route()`) i es retorna una resposta simulada, sense dependre de cap servidor ni base de dades real.

9.4.2 Cobertura i exemples representatius

La Taula 9.1 recull el nombre de proves i la cobertura de codi mesurats directament sobre l'estat actual del repositori.

Component	Proves	Fitxers	Cobertura (línies)
<i>Backend</i> (unitàries, Jest)	359	27	63,9 %
<i>Backend</i> (e2e, Supertest)	135	8	–
<i>Frontend</i> (unitàries, Vitest)	563	67	65,8 %
<i>Frontend</i> (e2e, Playwright)	19	2	–

TAULA 9.1: Nombre de proves i cobertura de codi, mesurats directament sobre el repositori.

La cobertura és el percentatge del codi que arriben a executar les proves. No es persegueix el 100 % (suposaria moltíssima feina per poc benefici); n'hi ha prou que les funcionalitats necessàries quedin cobertes, de manera que un canvi futur que en trenqui alguna dispari un error visible en comptes de passar desapercebut.

9.5 Codi font i vídeo

En aquest repositori públic de GitHub, hi trobareu (a data de publicació del projecte i fins a la seva defensa), tot el codi font així com el vídeo de demostració de la plataforma: github.com/raamonsiu/LSIGNER_PUBLIC.

Capítol 10

Implantació i resultats

El disseny (Capítol 8), la implementació (Capítol 9) i la metodologia (Capítol 3) ja s'han explicat des d'una perspectiva d'enginyeria. Aquest capítol tanca el cicle des de la perspectiva del procés real d'implantació: com es va passar, pas a pas, del primer *commit* a un sistema desplegat i funcionant en un servidor real. La segona part avalua el grau d'assoliment dels objectius (Capítol 1) i dels requisits (Capítol 6), i revisa el compliment de la normativa identificada a la viabilitat legal (Capítol 2).

10.1 Procés d'implantació

El projecte va començar amb una etapa d'investigació del marc teòric, el mercat i les tecnologies (Capítols 5, 2 i 7), a partir de la qual es va dissenyar la base del sistema: el model de dades, els casos d'ús i l'arquitectura (Capítol 8), sota la metodologia àgil adaptada descrita al Capítol 3. Només llavors va començar l'etapa d'implementació.

10.1.1 Primers passos: el *backend*

El desenvolupament es va iniciar pel *backend*: instal·lació de Node.js, eines de control i qualitat de codi, i configuració inicial de NestJS. Un cop llest, es va connectar a PostgreSQL en local i es van aplicar les primeres migracions de TypeORM. Amb el *backend* avançant en local, es va contenidoritzar, ja pensant en el desplegament futur (Secció 9.2.4), i en paral·lel es van introduir les primeres *pipelines* d'integració contínua (Capítol 3).

10.1.2 Aixecament de la infraestructura de producció

Un cop contenidoritzat el *backend*, calia decidir on s'executaria en producció: es va optar per un pla KVM2 de Hostinger (comparativa justificada al Capítol 7) i es va comprar el domini. El primer pas va ser administratiu: donar-se d'alta, contractar el servei i configurar els DNS dels subdominis de l'API i del *frontend* cap al VPS, encara sense el *proxy* invers (Traefik). Tot seguit es va configurar el servidor: un usuari dedicat de desplegament, tots els ports tancats excepte SSH i HTTPS, bloqueig d'adreces IP davant d'intents fallits per SSH, i còpies de seguretat de Hostinger complementades amb unes altres generades des de dins del mateix servidor.

Amb el servidor preparat, es va instal·lar Dokploy (Secció 7.4) i configurar els dos primers serveis: PostgreSQL i el *backend*. El *backend* es va vincular a una GitHub App pròpia, amb els secrets incrustats directament al servei. PostgreSQL es va configurar a partir de la plantilla de Dokploy amb la mateixa versió que en local, però amb

credencials noves i pròpies de producció, amb accés intern des del *backend* i accés extern d'administració via Tailscale, instal·lat i configurat de zero (l'única dificultat va ser restringir la ruta perquè només redirigís al port de PostgreSQL, i no a tota la xarxa).

Un cop actius els dos serveis, es van configurar els dominis dins de Dokploy i la infraestructura va quedar llesta. Garage (Secció 7.4) es va afegir més endavant, ja amb tot funcionant, com a destinació de les còpies de seguretat de la BD. Durant aquest procés es va acabar també el *backend*, incloent-hi la documentació de l'API, i el projecte va passar a centrar-se en el *frontend*.

10.1.3 Desenvolupament i desplegament del *frontend*

El *frontend* va seguir un procés similar: configuració inicial de Next.js i contenidorització gairebé immediata, ja que l'experiència del *backend* es reaprofitava directament. La diferència més notable va aparèixer en configurar-ne el servei a Dokploy: la política CORS entre dominis diferents va donar més d'un problema abans d'ajustar-la correctament.

10.1.4 El proveïdor de correu electrònic

Amb la plataforma ja funcionant, i mentre el desenvolupament del *frontend* continuava avançant, va tocar trobar un proveïdor de correu per als OTP i les notificacions de documents (Secció 9.2.2); l'opció de Hostinger resultava massa cara per a l'escala del projecte. Després d'una cerca extensa, es va optar per MXRoute, un servei gestionat per una única persona als Estats Units: en una videotrucada amb el propietari, quan se li va exposar l'abast i l'ús previst del projecte, va oferir un preu encara més ajustat que el ja econòmic que anunciava, agraït per l'interès mostrat en el seu servei. El cost final es recull al pressupost del Capítol 2.

10.1.5 Tancament: proves reals i redacció de la memòria

Un cop el sistema complet estava desplegat, la resta del temps es va dedicar a finalitzar el desenvolupament pendent, fer proves reals sobre la plataforma ja funcionant i redactar aquesta memòria. No totes les funcionalitats previstes a l'inici es van arribar a completar: la migració dels fitxers binaris a l'emmagatzematge S3, la pàgina de seguretat del *frontend* i el suport de multifactor o claus personals de signatura van quedar fora de l'abast final, tal com ja s'explica al Capítol 3.

10.2 Compliment legal i normatiu

El Capítol 2 ja va identificar els quatre marcs normatius rellevants per al projecte (eIDAS, RGPD, LSSICE i els principis de l'ENS) i en va deixar el compliment concret pendent d'aquest capítol.

eIDAS. Tal com es va decidir a la viabilitat legal, LSigner s'emmarca únicament en la signatura electrònica simple (SES), reforçada amb evidències orientades al no-repudi: l'artefacte Ed25519 sobre el *payload* canònic, la cadena de custòdia i l'historial d'accions (Secció 9.2.1). Aquest era l'únic nivell que el projecte es proposava assolir, i s'ha complert íntegrament.

RGPD. El compliment es recolza en mecanismes reals, no només declaratius. Primer, la minimització de dades i la retenció: un procés automàtic (**PurgeCronService**) s'executa diàriament i elimina els registres que superen el període de conservació establert de 12 mesos, el mateix que fixa el RF13 del Capítol 6. Segon, el dret a l'oblit: eliminar el compte no es limita a marcar-lo com a esborrat, sinó que anonimitza realment les dades identificatives (nom, cognoms, correu electrònic i telèfon) abans de deixar-lo pendent de purga definitiva. Tercer, la informació a l'usuari: la plataforma publica el conjunt de documents legals detallat a l'apartat de la LSSICE, disponibles en els tres idiomes de la plataforma.

Cal fer una precisió respecte de la viabilitat legal, on es parlava de mantenir xifrades les dades sensibles. Les contrasenyes no es xifren, sinó que es deriven amb *scrypt* i una sal aleatòria pròpia per usuari: la pràctica correcta, ja que no s'han de poder recuperar ni tan sols des del mateix sistema. La resta de dades personals (com el telèfon), en canvi, es guarden en clar, protegides només pel control d'accés (autenticació per defecte, aïllament de xarxa de la base de dades, secrets fora del codi), no per xifratge de camp. És una diferència real respecte del plantejament inicial, d'impacte reduït donat l'abast del projecte.

LSSICE. La pàgina de documents legals no es limita al mínim habitual d'un lloc web (avís legal, privacitat i *cookies*), sinó que inclou també els documents propis d'una plataforma de signatura electrònica: la política de signatura electrònica, que regula sota quines condicions es considera vàlida una signatura a la plataforma; l'acord de tractament de dades, per a un ús futur com a encarregat del tractament davant de clients B2B; i la política de conservació de documents signats.

A banda d'aquests documents, la LSSICE exigeix que l'usuari conegui, i si cal consenti, l'ús de *cookies*. LSigner només en fa servir de tècniques (*session_id*, per mantenir la sessió) i funcionals (tema, idioma), sense cap *cookie* analítica ni de tercers; el sistema d'analítica interna tampoc fa servir emmagatzematge al navegador. Segons l'article 22 de la LSSICE, les *cookies* estrictament necessàries per al servei sol·licitat no requereixen consentiment previ, de manera que no calia cap bàner, més enllà de l'avís que ja recull la mateixa política de *cookies*.

ENS. Els seus principis es prenen només com a referència de bones pràctiques (Capítol 2), i el seu compliment es reflecteix en les mateixes mesures de seguretat ja avaluades al requisit RNF1 del Capítol 6.

10.3 Assoliment dels objectius

El Capítol 1 definia els objectius del projecte en dos blocs (Secció 1.4): els de procés i els de producte.

Els objectius de procés s'han assolit tots. S'han aplicat pràctiques de codi net i patrons de disseny (Capítols 8 i 9); s'ha treballat sota una metodologia àgil adaptada, amb seguretat i pràctiques DevOps integrades des del primer dia (Capítol 3); s'ha dissenyat i configurat una cadena completa d'integració i desplegament continu, amb contenidorització Docker i desplegament en un VPS real (Secció 10.1); s'ha experimentat el cicle de vida complet d'un producte propi, des del primer *commit* fins a la posada en producció real; i s'han dut a terme amb èxit totes les integracions

previstes amb serveis externs (domini, VPS, plataforma de desplegament i correu electrònic).

Els objectius de producte també es consideren assolits, amb un matís. La plataforma ha arribat a una qualitat de producció, amb una arquitectura modular i mantenible (Capítol 8); la signatura de documents assoleix un nivell de seguretat robust i verificable de manera independent, complint la normativa vigent (Secció 10.2); i el producte final està desplegat i en funcionament en un entorn de producció real i estable, amb el cicle de desplegament continu descrit en aquest capítol. El matís correspon al mateix que ja apuntava la definició adaptativa de l'abast (Capítol 3): no totes les funcionalitats previstes inicialment s'han arribat a completar, tot i que cap d'aquestes no compromet els objectius de producte fixats a l'inici del projecte.

10.4 Assoliment dels requisits

La Taula 6.1 (Capítol 6) ja indica l'estat de cada requisit funcional; només cal aturar-se en els dos que hi queden marcats com a parcials. El RF10 (bloqueig d'accés a documents) té la lògica completa implementada al *backend*, però la seva integració a la interfície del *frontend* ha quedat pendent. El RF11 (accés de destinataris externs) es troba en un punt similar: el flux complet (sessió pública temporal, *cookie* segura i el mateix procés de signatura que un usuari registrat) ja funciona íntegrament al *backend* (Apèndix C), però encara no s'ha integrat a les pantalles del *frontend*. En els dos casos, la part de disseny i de lògica de negoci ja és sòlida, i només falta la darrera capa d'interfície per exposar-la a l'usuari.

Pel que fa als requisits no funcionals (Taula 6.2), la majoria s'han assolit, tal com ja es detalla al llarg d'aquest capítol i del Capítol 9, tot i que dos queden parcials. El RNF6 (preparació per a l'escalabilitat) es limita a una arquitectura modular pensada per créixer: serveis independents, contenidors, base de dades relacional ben normalitzada; però el projecte no ha arribat a validar cap escalat real, ja que no ha calgut. El RNF7 (usabilitat i accessibilitat) es recolza en una biblioteca de components (MUI) que ja segueix les pautes WAI-ARIA de sèrie; a més, s'han fet proves manuals d'accessibilitat, replicant el flux principal de signatura només amb teclat, sense ratolí. No s'ha dut a terme, però, cap auditoria d'accessibilitat formal (amb un lector de pantalla o una eina automàtica) sobre la interfície final.

El RNF8 (rendiment) sí que es pot validar amb mètriques reals, mesurades directament sobre l'entorn de desenvolupament desplegat a Dokploy: una petició a l'API respon en uns 150 ms un cop el servei ja ha atès alguna petició prèvia, i el *frontend* respon en 150–230 ms un cop la ruta sol·licitada ja s'ha carregat un primer cop. La primera petició a cada ruta és sensiblement més lenta, al voltant d'1 segon, probablement pel cost de càrrega inicial dels seus mòduls a memòria; les peticions posteriors a la mateixa ruta, ja amb el codi carregat, es resolen en el rang anterior.

Capítol 11

Conclusions

El disseny i la implementació del sistema (Capítols 8 i 9) i el grau d'assoliment dels objectius i els requisits (Capítol 10) ja s'han detallat capítol a capítol. El que segueix no repeteix aquella anàlisi: en fa una síntesi, hi afegeix les desviacions respecte de la planificació inicial (Capítol 4) i tanca amb les aportacions personals i l'aprenentatge que n'he tret.

11.1 Valoració global

La valoració general del projecte és molt positiva. Els objectius definits a l'inici (Secció 1.4), tant els de procés com els de producte, s'han assolit en la seva pràctica totalitat, tal com ja s'ha detallat al capítol anterior (Secció 10.3); el mateix val per als requisits del sistema (Capítol 6), amb només quatre excepcions parcials, ja raonades, que no comprometen cap objectiu de fons. El resultat és una plataforma de signatura electrònica completa, desplegada i en funcionament real, que compleix el propòsit amb què va néixer el projecte (Capítol 1). Es tracta, a més, d'un cicle complet: des de la primera línia de codi fins a un sistema real, desplegat i mantingut en producció (Capítol 10), fet en solitari i sota una metodologia àgil adaptada (Capítol 3).

Aquest capítol no inclou una comparativa amb altres plataformes de signatura electrònica existents al mercat. Com ja s'establia al propòsit del projecte (Capítol 1), l'objectiu mai va ser oferir la millor aplicació possible, sinó prioritzar la qualitat d'enginyeria i l'aprenentatge que se'n deriva; una comparativa d'aquest tipus no aportaria, doncs, valor real a aquestes conclusions (tot i que ja us podria avançar el resultat de comparar LSigner amb alguna *castanya* de les que hi ha al mercat, si em permeteu el col·loquialisme).

11.2 Principals resultats

Des d'un punt de vista més tècnic, la plataforma s'ha mantingut operativa de manera contínua i sense caigudes des que la infraestructura de producció va quedar desplegada (Secció 10.1), amb els temps de resposta ja mesurats al capítol anterior (Secció 10.4). Tots els serveis, el *backend*, el *frontend*, la base de dades i l'emmagatzematge d'objectes, funcionen de manera estable i coordinada sobre el mateix servidor.

El comportament de la interfície també s'ha comprovat manualment en els navegadors d'escriptori més habituals, verificant tant la consistència visual com els fluxos

principals (enviament, recepció i signatura de documents), amb resultats homogenis. Queda pendent, en canvi, l'adaptació a mòbil, que no ha estat possible ni de manera responsiva al navegador ni com a aplicació progressiva o nativa, una part que el projecte no ha arribat a cobrir dins de l'abast final.

Pel que fa a la infraestructura i la seguretat, el balanç és igualment sòlid. El desplegament continu amb possibilitat de *rollback*, les còpies de seguretat periòdiques de la base de dades i les mesures de seguretat integrades des del disseny (RNF1 i RNF5, Capítol 6) han funcionat exactament com es van dissenyar, sense cap incident destacable durant tot el període d'implantació. És, probablement, l'àrea on la diferència entre el plantejament inicial i el resultat final és més petita: el que es va decidir a la planificació i a l'estudi de tecnologies (Capítols 4 i 7) es correspon gairebé exactament amb el que hi ha desplegat a hores d'ara.

11.3 Desviacions de la planificació

El cronograma real (Figura 4.1, Capítol 4) no va seguir de manera estricta els paquets de treball previstos, tot i que totes les desviacions rellevants responen a motius lògics o de dependència, i no a una mala estimació.

Primer, el correu electrònic: tot i pertànyer al paquet d'infraestructura, la seva configuració no es va abordar fins a la setmana 11, coincidint amb l'inici del *frontend* i no amb la resta de tasques d'infraestructura o *backend*; el flux d'OTP del *backend* resultava més fàcil d'implementar un cop el *frontend* ja hi pogués interactuar, de manera que aquesta part es va ajornar deliberadament. Segon, el desplegament del servei de *frontend* a Dokploy, que, tot i pertànyer també al mateix paquet, no arriba fins a les darreres setmanes del *frontend*: és una dependència natural, ja que no té sentit desplegar un servei que encara no és funcional. Tercer, l'emmagatzematge S3 i les còpies de seguretat que en depenen, situats al tram final de tot el cronograma; aquí el motiu no és de dependència sinó de prioritat, ja que la necessitat d'un emmagatzematge d'objectes propi es va identificar més tard del previst inicialment. En conjunt, les tres desviacions confirmen que el doble nivell de planificació, per paquets de treball i per iteracions curtes (Capítol 4), va absorbir bé els canvis de prioritat, sense necessitat de cap replanificació dràstica.

11.4 Aportacions personals i aprenentatge

Com a aportació personal, destaco haver dissenyat, desenvolupat i desplegat en solitari un sistema complet: les tres capes (*backend*, *frontend* i infraestructura) i tots els processos que les envolten (control de versions, integració contínua, desplegament, seguretat), sense cap suport tècnic extern més enllà de la tutoria periòdica (Capítol 3). Això ha implicat prendre, de manera individual, totes les decisions d'arquitectura i de tecnologia que en un entorn professional se solen repartir entre diversos perfils especialitzats (Capítol 7): des del disseny del model de dades fins a la tria del proveïdor de correu electrònic, passant per la seguretat de la cadena de signatura i la configuració de la xarxa del servidor.

A nivell d'aprenentatge, aquest projecte m'ha aportat, per damunt de tot, una lliçó d'humilitat professional. Vaig començar les pràctiques laborals a finals del tercer

curs i, en plantejar aquest projecte, tenia la sensació d'haver après ja molt de desenvolupament web. Desenvolupar íntegrament LSigner m'ha demostrat que no era així: la informàtica és un camp extraordinàriament ampli i, tot i no haver sortit en cap moment de l'àmbit del desenvolupament web, hi ha hagut molts temes on he après moltíssim més del que pensava que ja sabia.

Destaca, sobretot, tot el que envolta la integració i el desplegament continu i la infraestructura (Capítol 3 i Secció 10.1): configurar un servidor de zero, encadenar processos de *build*, prova i desplegament de manera automàtica, o aïllar una base de dades amb una xarxa privada, són coses que a la feina havia vist fer a altres persones, però que no havia arribat a fer jo mateix de principi a fi. Hi sumo les integracions amb serveis externs (domini, correu electrònic, emmagatzematge d'objectes) i totes les particularitats reals d'un sistema en producció, temes que ni la carrera ni la feina m'havien permès tocar amb tant de detall. Si l'experiència professional prèvia em va donar una base sòlida en el disseny de programari, aquest projecte n'hi ha afegit una altra, igual d'important, sobre tot allò que cal perquè aquest programari arribi a funcionar de veritat i en entorns de producció.

11.5 Reflexió final

Finalment, no m'agradaria tancar la memòria sense un apunt més honest o sentimental. Malgrat el balanç molt positiu, hi ha coses que m'hauria agradat completar i que el temps no ha permès (Secció 10.1); en especial, la pàgina de seguretat del *frontend*, on l'usuari hauria pogut comprovar i verificar les signatures dels seus documents directament des de la mateixa aplicació. La capacitat de verificació independent ja existeix (RF8, Capítol 6), però només com a *endpoint* de *backend*; no arribar a exposar-la en una pantalla pròpia és, probablement, el que trobo més a faltar del resultat final. És, en tot cas, una feina pendent ben acotada i no pas un problema de fons: la base sobre la qual construir-la (Capítol 9) ja hi és.

Capítol 12

Treball futur

LSigner arriba, amb aquest projecte, a un estat estable, funcional i desplegat en un entorn de producció real (Capítol 10), però això no vol dir que el seu desenvolupament s'hagi tancat. Aquest capítol en recull les línies de treball futur, organitzades en tres blocs de naturalesa diferent: primer, la feina tècnica que ja s'havia identificat durant el projecte però que en va quedar fora de l'abast final (Capítol 3); segon, noves línies d'evolució que ampliarien l'abast actual de la plataforma; i, finalment, què implicaria portar LSigner més enllà d'un projecte acadèmic i plantejar-lo com un producte real.

12.1 Treball pendent

Alguns dels requisits que es van deixar deliberadament fora de l'abast final (Capítols 3 i 10) continuen sent, de fet, la feina pendent més immediata.

Emmagatzematge de documents a S3. Actualment, els fitxers binaris dels documents es guarden directament a PostgreSQL; migrar-los a un *bucket* S3 alliberaria la base de dades d'aquesta càrrega i n'oferiria un escalat molt més natural. La infraestructura necessària ja hi és: Garage (Capítol 7) ja s'emptra per a les còpies de seguretat (Secció 10.1); només caldria estendre'n l'ús a l'emmagatzematge actiu i adaptar la capa d'accés del *backend* perquè hi llegeixi i hi escrigui directament.

Pàgina de seguretat del *frontend*. Tal com es lamentava a la reflexió final (Capítol 11), l'*endpoint* de verificació independent (RF8, Capítol 6) ja funciona; falta només exposar-lo en una pantalla pròpia del *frontend* que permeti a qualsevol usuari comprovar la validesa d'una signatura sense sortir de l'aplicació.

Autenticació multifactor i claus personals de signatura. El sistema identifica el signant amb contrasenya i un OTP per correu electrònic (RF6, Capítol 6); incorporar-hi un segon factor addicional (una aplicació autenticadora, per exemple) o una clau criptogràfica pròpia de cada usuari reforçaria encara més les barreres de seguretat descrites al Capítol 1.

Middleware intern d'anàlisi de comportament. Tota la seguretat actual del *backend* és de caràcter preventiu: autenticació, validació i anàlisi estàtica de codi a la integració contínua (Capítol 3). Hi manca, en canvi, una capa de detecció activa: un *middleware* intern que analitzi patrons de comportament de les peticions (intents d'OTP repetidament fallits, o un volum de signatures anòmal per a un mateix compte, per

exemple) i que alerti o actuï davant d'un comportament sospitos, més enllà del control d'accés estàtic que ja ofereix el sistema (Capítol 8).

12.2 Noves línies d'evolució

Més enllà de tancar la feina pendent, hi ha diverses direccions que ampliarien l'abast actual de la plataforma.

Alta disponibilitat. El desplegament actual sobre un únic servidor virtual és, per disseny, un punt únic de fallada (Capítol 2); introduir-hi redundància, rèpliques de base de dades i balanceig de càrrega en faria un sistema molt més resiliència, tot i que a un cost d'infraestructura molt superior al d'aquest projecte.

Adaptació mòbil. LSigner s'ha provat i funciona correctament en múltiples navegadors d'escriptori (Capítol 10), però mai s'ha arribat a adaptar de manera responsiva, ni com a aplicació web progressiva ni com a aplicació nativa (Capítol 11). Signar documents des del mòbil és, probablement, un dels casos d'ús més habituals en un producte d'aquest tipus, de manera que aquesta adaptació seria una de les properes prioritats naturals.

Integració B2B. L'orientació B2B es va deixar deliberadament oberta des de l'inici (Capítol 3), i la plataforma ja preveu part del terreny legal per fer-hi el pas (Secció 10.2). Adaptar LSigner perquè altres empreses el puguin adoptar en mode d'integració (per exemple, incrustat mitjançant *iframe* o consumit directament via API, tal com ja fan proveïdors similars del sector, Capítol 2), acompanyat d'una documentació d'integració que en faciliti l'adopció, formalitzaria aquesta línia. Això implicaria, també, resoldre la mancança de rols organitzatius ja identificada (Capítol 8).

Signatura basada en *blockchain*. És, irònicament, de les quatre, la línia menys madura i la que exigiria més disseny previ. Es descarta, d'entrada, una xarxa *blockchain* pròpia i descentralitzada entre els mateixos usuaris de la plataforma: un disseny així exigiria una base d'usuaris molt àmplia i permanentment activa per sostenir el consens, cosa que no té sentit per a una plataforma de signatura de documents. L'alternativa realista seria integrar-se amb una xarxa *blockchain* ja existent i consolidada, i construir-hi a sobre una capa de contractes intel·ligents que ancori les evidències de signatura de manera immutable. L'antecedent professional amb aquesta tecnologia (Capítols 1 i 5) ja aporta bona part del coneixement necessari per abordar-ho.

12.3 Cap a un producte real

Tot el que s'ha comentat fins ara és una evolució natural de la plataforma tal com existeix avui. Convertir LSigner en un producte real, més enllà d'un projecte acadèmic, exigiria abordar precisament la barrera que l'estudi de viabilitat ja va identificar com la més limitant (Secció 2.4): assolir els nivells de signatura avançada (AES) o qualificada (QES) implica treballar amb prestadors de certificats acreditats i dispositius segurs de creació de signatura, i probablement certificacions de seguretat de la informació (com ara ISO 27001 [19]) si es vol oferir garanties a clients B2B o optar a processos de licitació pública.

Aquest salt només té sentit si hi ha, al darrere, un model de negoci real que en justifiqui la inversió: ja sigui mitjançant una subscripció o facturació per signatura orientada a clients *B2B* (com ja fan els proveïdors comentats al Capítol 2), ja sigui posicionant el producte per a processos de licitació pública, amb els seus propis requisits de compliment i certificació. És, en definitiva, la diferència entre demostrar que una base tècnica sòlida és possible, que és l'objectiu d'aquest projecte (Capítol 1), i construir-hi al voltant un producte i un negoci viables.

Apèndix A

Fitxes de casos d'ús

Aquest annex recull les fitxes dels casos d'ús més rellevants del sistema, referenciats des del Capítol 8. Les accions de signar, rebutjar i revocar comparteixen la verificació d'identitat mitjançant OTP, de manera que s'agrupen per evitar repeticions.

CU-01 Enviar document

Objectiu	Fer arribar un document a un o més destinataris perquè el signin.
Actor	Usuari registrat (emissor).
Precondicions	L'usuari està autènticat.
Flux principal	1. L'usuari puja el document al sistema o en selecciona un de ja existent. 2. Selecciona l'opció d'enviar-lo. 3. Afegeix un o més destinataris (contactes o adreces externes). 4. Opcionalment, hi estableix una contrasenya d'accés. 5. Confirma l'enviament. 6. El sistema crea les línies de destinatari, marca el document com a enviat i notifica els destinataris per correu electrònic.
Fluxos alternatius	1a. El fitxer no supera la validació (format o mida): el sistema no l'accepta i n'informa l'usuari. 3a. Un destinatari no té compte: rebrà un enllaç públic per signar. 6a. Falla l'enviament d'un correu: el sistema ho registra i n'informa l'emissor.
Postcondicions	El document queda pendent de signatura i els destinataris en reben la notificació.

CU-02 Resoldre un document rebut (signar o rebutjar)

Objectiu	Signar o rebutjar un document rebut, deixant-ne una evidència verificable.
Actors	Usuari registrat o destinatari extern (signant).
Precondicions	El signant ha rebut un document pendent i hi té accés (sessió autenticada o enllaç públic vàlid).
Flux principal	1. El signant accedeix al document pendent. 2. El sistema li envia un OTP per correu i li'n demana la introducció. 3. El signant introdueix l'OTP i decideix signar el document. 4. El sistema valida l'OTP, genera l'artefacte de signatura Ed25519 sobre el <i>payload</i> canònic i enregistra l'evidència i l'esdeveniment a l'historial.
Fluxos alternatius	1a. El document està bloquejat: el signant ha de resoldre el bloqueig (introduir la contrasenya d'accés) abans de continuar. 3a. Rebuig: el signant rebutja el document; el sistema en registra el rebuig i l'esdeveniment, sense generar cap signatura. 4a. OTP incorrecte o caducat: el sistema rebutja el codi i permet reenviar-ne un de nou, amb un límit d'intents.
Postcondicions	El document queda signat (amb una evidència verificable de manera independent) o rebutjat, i l'acció consta a l'historial immutable.

CU-03 Revocar una signatura

Objectiu	Retirar una signatura emesa prèviament, deixant-ne constància a l'historial.
Actors	Usuari registrat o destinatari extern (signant).
Precondicions	El signant ha signat el document.
Flux principal	1. El signant accedeix al document signat i selecciona revocar-ne la signatura. 2. El sistema li envia un OTP per correu i li'n demana la introducció. 3. El signant introdueix l'OTP. 4. El sistema valida l'OTP, marca la signatura com a revocada i registra l'esdeveniment corresponent a l'historial.
Fluxos alternatius	1a. La signatura ja no és revocable: el sistema n'informa i no fa cap canvi. 4a. OTP incorrecte o caducat: es comporta com en el cas CU-02.
Postcondicions	La signatura queda revocada, l'acció consta a l'historial i el document ja no admet cap més acció per part d'aquest usuari.

CU-04 Verificar una signatura de manera independent

Objectiu	Permetre a qualsevol tercer comprovar la validesa d'una signatura sense dependre de la disponibilitat ni de la confiança en la plataforma.
Actor	Qualsevol tercer (no cal ser usuari registrat ni destinatari del document).
Precondicions	El document té almenys un destinatari amb una signatura emesa.
Flux principal	1. El tercer accedeix a l'<i>endpoint</i> públic de verificació indicant l'identificador del document i del destinatari. 2. El sistema retorna la signatura Ed25519, la clau pública, el <i>payload</i> canònic signat i el resultat de la verificació calculada pel servidor mateix. 3. El tercer pot copiar aquestes dades i verificar-les de manera independent amb qualsevol eina Ed25519 estàndard, sense dependre del resultat que retorna el sistema.
Fluxos alternatius	1a. No existeix cap artefacte signat per a la combinació de document i destinatari indicada: el sistema respon amb un error 404.
Postcondicions	El tercer disposa de l'evidència criptogràfica i de la seva verificació, sense que calgui autenticar-se ni exposar el contingut del document.

CU-05 Accedir com a destinatari extern

Objectiu	Permetre a un destinatari sense compte accedir al document que se li ha enviat, sense necessitat de registrar-se.
Actor	Destinatari extern.
Precondicions	El destinatari ha rebut un enllaç públic d'accés a un document pendent.
Flux principal	1. El destinatari obre l'enllaç públic rebut per correu electrònic. 2. El sistema inicia una sessió pública temporal associada a aquell destinatari, sense demanar-li cap credencial. 3. El destinatari consulta el document i, si escau, en resol els bloquejos d'accés. 4. A partir d'aquí segueix el mateix flux de resolució que un usuari registrat (fitxa CU-02).
Fluxos alternatius	1a. L'enllaç ha caducat o el document ja no és accessible: el sistema n'informa el destinatari i no inicia sessió.
Postcondicions	El destinatari disposa d'accés temporal al document sense necessitat de crear cap compte.

Nota d'estat: aquest darrer cas d'ús correspon al RF11 (Capítol 6), amb el flux complet implementat al backend i la integració al frontend encara parcial.

Apèndix B

Disseny detallat de la base de dades

Aquest annex completa el disseny de la base de dades presentat al Capítol 8 (Secció 8.2.1) amb la resta de taules del sistema.

TAULA B.1: Disseny de la taula `users`.

Camp	Tipus	Clau	Descripció
<code>patient_id</code>	<code>uuid</code>	PK	Identificador de l'usuari.
<code>name/last_name</code>	<code>varchar</code>		Nom i cognoms.
<code>country</code>	<code>varchar(100)</code>		País de residència.
<code>national_id</code>	<code>varchar(50)</code>		Document d'identitat, opcional i únic.
<code>passport</code>	<code>varchar(50)</code>		Passaport, opcional i únic.
<code>email</code>	<code>varchar</code>		Correu electrònic, únic; s'anonimitza en esborrar el compte.
<code>phone_number</code>	<code>varchar(30)</code>		Telèfon en format E.164, opcional.
<code>deleted_at</code>	<code>timestampz</code>		Marca de temps de l'eliminació (<i>soft-delete</i>); nul si el compte és actiu.
<code>password / salt</code>	<code>varchar</code>		Clau derivada (scrypt) i sal; excloses de les consultes per defecte.
<code>created_at / updated_at</code>	<code>timestampz</code>		Marques de temps.

TAULA B.2: Disseny de la taula `contacts`.

Camp	Tipus	Clau	Descripció
<code>id</code>	<code>uuid</code>	PK	Identificador del contacte.
<code>owner_id</code>	<code>uuid</code>	FK <i>users</i>	Usuari propietari de l'agenda; esborrat en cascada.
<code>contact_user_id</code>	<code>uuid</code>	FK <i>users</i>	Usuari registrat referenciat, si el contacte en té; queda a nul si aquest usuari s'elimina.
<code>contact_email</code>	<code>varchar(255)</code>		Correu del contacte.

Camp	Tipus	Clau	Descripció
contact_name	varchar(200)		Nom opcional.
contact_phone	varchar(30)		Telèfon opcional.
created_at/up dated_at	timestampz		Marques de temps.

Restricció d'unicitat: (*owner_id*, *contact_email*).

TAULA B.3: Disseny de la taula *refresh_tokens*.

Camp	Tipus	Clau	Descripció
id	uuid	PK	Identificador del <i>token</i> .
user_id	uuid	FK <i>users</i>	Usuari propietari; esborrat en cascada.
token_hash	char(64)		Empremta SHA-256 del <i>token</i> opac; el valor en clar mai es persisteix.
expires_at	timestampz		Data de caducitat.
revoked	boolean		Cert un cop el <i>token</i> s'ha rotat o invalidat.
created_at	timestampz		Marca de temps de creació.

TAULA B.4: Disseny de la taula *document_locks*.

Camp	Tipus	Clau	Descripció
id	uuid	PK	Identificador del bloqueig.
document_id	uuid	FK <i>documents</i>	Document protegit; esborrat en cascada.
lock_type	enum		Tipus de bloqueig; actualment només PASSWORD .
config	jsonb		Configuració específica del tipus (per contrasenya: clau derivada i sal); exclosa de les consultes per defecte.
created_at/up dated_at	timestampz		Marques de temps.

TAULA B.5: Disseny de la taula *document_lock_resolutions*.

Camp	Tipus	Clau	Descripció
id	uuid	PK	Identificador de la resolució.
lock_id	uuid	FK <i>document_locks</i>	Bloqueig resolt; esborrat en cascada.
recipient_id	uuid	FK <i>document_recipients</i>	Destinatari que l'ha resolt; esborrat en cascada.

Camp	Tipus	Clau	Descripció
resolved_at	timestampz		Moment en què es va resoldre.
created_at	timestampz		Marca de temps de creació.

Restricció d'unicitat: (lock_id, recipient_id); evita que un mateix destinatari resolgui dues vegades el mateix bloqueig.

TAULA B.6: Disseny de la taula otp_challenges.

Camp	Tipus	Clau	Descripció
id	uuid	PK	Identificador del repte.
user_id	uuid		Usuari que ha de verificar-se (sense relació d'ORM; vegeu Secció 8.1.4).
action_type	enum		Acció que motiva el repte: SIGN / REJECT / REVOKE.
resource_type / resource_id	enum / uuid		Tipus i identificador del recurs afectat; avui sempre un document.
otp_hash/otp_salt	varchar		Codi hashejat i sal; exclosos de les consultes per defecte.
expires_at	timestampz		Caducitat del codi (300 s per defecte).
attempt_count / max_attempts	int		Intents consumits i màxim permès (5 per defecte).
resend_count / max_resends	int		Reenviaments consumits i màxim permès (3 per defecte).
resend_available_at	timestampz		Moment a partir del qual es permet un nou reenviament; nul si no n'hi ha cap en curs.
locked_until	timestampz		Bloqueig temporal després d'exhaurir els intents (15 min per defecte).
status	enum		Estat del repte (actiu, verificat, caducat...).
metadata	jsonb		Dades addicionals de context de la sol·licitud.
created_at / updated_at	timestampz		Marques de temps.

Índex compost sobre (user_id, action_type, resource_type, resource_id, status): accelera la cerca d'un repte actiu en crear-ne un de nou, no la verificació (que es fa sempre per id). Un índex únic parcial sobre el mateix subconjunt de camps garanteix, a més, que només pugui existir un repte amb estat actiu per a la mateixa combinació (usuari, acció, recurs).

TAULA B.7: Disseny de la taula `public_link_sessions`.

Camp	Tipus	Clau	Descripció
<code>id</code>	<code>uuid</code>	PK	Identificador de la sessió.
<code>recipient_id</code>	<code>uuid</code>	FK <i>document_recipients</i>	Destinatari extern associat; esborrat en cascada.
<code>session_hash</code>	<code>varchar(64)</code>		Empremta única de la sessió (la referència en clar només viatja en una <i>cookie</i>).
<code>expires_at</code>	<code>timestampz</code>		Caducitat de la sessió.
<code>last_used_at</code>	<code>timestampz</code>		Darrer ús, si escau.
<code>revoked_at</code>	<code>timestampz</code>		Moment de revocació, si escau.
<code>ip/user_agent</code>	<code>varchar</code>		Metadades de context de la sessió, opcionals.
<code>created_at/updated_at</code>	<code>timestampz</code>		Marques de temps.

Apèndix C

Detalls addicionals d'implementació

Aquest annex amplia dos detalls d'implementació del Capítol 9 que no calia desenvolupar dins del cos del capítol.

C.1 Sessions públiques per a destinataris externs

La Secció 9.2.3 explica com gestiona la sessió l'usuari registrat; un destinatari extern (sense compte) segueix un camí diferent, ja que no té cap JWT amb què autenticar-se. En obrir l'enllaç públic, el *backend* li crea una sessió anònima (`PublicLinkSession`), amb el mateix patró que els *refresh tokens* (Taula B.3, Apèndix B): es genera un valor aleatori de 32 bytes, i només se'n persisteix l'empremta a la base de dades; el valor en clar viatja únicament dins d'una *cookie* i mai s'escriu enlloc.

El punt d'entrada distingeix dos casos (no tots els destinataris d'un enllaç són externs) i revoca qualsevol sessió anterior del mateix destinatari abans de crear-ne una de nova, perquè només n'hi pugui haver una d'activa alhora:

LISTING C.1: *Bootstrap* d'una sessió pública
(`public-session.service.ts`)

```

1 if (recipient.user_id) {
2   return { status: 'AUTH_REQUIRED', documentId: recipient.document_id };
3 }
4
5 await entityManager.update(
6   PublicLinkSession,
7   { recipient_id: recipient.id, revoked_at: IsNull(), expires_at: MoreThan
8     (new Date()) },
9   { revoked_at: new Date() },
10 );
11
12 const sessionToken = crypto.randomBytes(32).toString('hex');
13 const sessionHash = this.hashSessionToken(sessionToken);
14
15 const session = entityManager.create(PublicLinkSession, {
16   recipient_id: recipient.id,
17   session_hash: sessionHash,
18   expires_at: new Date(Date.now() + this.getSessionCookieMaxAgeMs()),
19   ip: context.ip,
20   user_agent: context.userAgent,
21 });
22 await entityManager.save(session);

```

```
22  
23 return { status: 'ANON_ALLOWED', documentId: recipient.document_id,  
    sessionToken };
```

Si el destinatari té compte (`user_id` no nul), se li exigeix iniciar sessió normalment (`AUTH_REQUIRED`): la sessió pública només existeix per a qui no en té cap altra. El controlador, un cop rep el `sessionToken`, el desa en una *cookie* marcada `httpOnly` (inaccessible des de JavaScript), `secure` en producció i amb `sameSite: 'lax'`; el valor en clar ja només viatja dins d'aquesta *cookie*.

En cada petició posterior, el sistema torna a validar la sessió (a partir de l'empremta de la *cookie*) abans de deixar-la passar: comprova que no s'hagi revocat ni caducat, i que el document i la signatura del destinatari continuïn en un estat que permeti l'accés (no s'hagi revocat la signatura ni el document s'hagi eliminat). És, doncs, el mateix nivell d'exigència que per a un usuari autenticat amb JWT, però verificat a cada petició en comptes de confiar en un *token* amb caducitat pròpia.

C.2 El symlink de Chromium a Alpine per a Playwright

Aquest apartat amplia la Secció 9.2.4: el cas de la imatge de desenvolupament del *frontend* (`Dockerfile.dev`), que necessita Chromium per executar les proves *end-to-end* de Playwright dins d'un contenidor.

El problema és que Playwright descarrega el seu propi binari de Chromium, compilat per a `glibc` (la biblioteca C d'Ubuntu i la majoria de distribucions d'escriptori). Les imatges del projecte es basen en Alpine, que fa servir `musl` en comptes de `glibc`: aquell binari, simplement, no hi arrenca.

La causa exacta és més aviat un problema de coordinació. Alpine sí que ofereix el seu propi paquet de Chromium, compilat per a `musl` i, per tant, compatible. El problema no és, doncs, la disponibilitat, sinó que Playwright no el busca allà: espera trobar el seu propi binari descarregat a un camí molt concret dins la seva memòria cau (`chromium_headless_shell-1228`, on 1228 identifica la revisió exacta que la versió instal·lada de `@playwright/test` espera).

La solució, doncs, és tan simple com instal·lar el Chromium d'Alpine i enganyar Playwright perquè el trobi, amb un enllaç simbòlic exactament a aquell camí:

LISTING C.2: El *symlink* que fa creure a Playwright que ha trobat el seu Chromium (`Dockerfile.dev`)

```
1 # Install system Chromium (compiled for musl, compatible with Alpine)  
2 RUN apk add --no-cache chromium  
3  
4 # Playwright v1.61.0 expects chromium_headless_shell-1228 at this exact  
5 # path  
6 # Symlink Alpine's Chromium so Playwright finds it at its expected  
7 # location  
8 RUN mkdir -p /root/.cache/ms-playwright/chromium_headless_shell-1228/  
    chrome-headless-shell-linux64 && \  
    ln -sf /usr/lib/chromium/chromium \  
    /root/.cache/ms-playwright/chromium_headless_shell-1228/chrome-  
    headless-shell-linux64/chrome-headless-shell
```

Com que `playwright.config.ts` no defineix cap `channel` ni `executablePath` propis, tot depèn que aquest camí exacte existeixi: per això el número de revisió (1228) ha d'anar lligat a la versió instal·lada de Playwright (canviaria en actualitzar-la), i per això la solució queda aïllada a la imatge de desenvolupament, sense afectar en cap cas la de producció.

Bibliografia

- [1] Parlament Europeu i Consell de la Unió Europea. *Reglament (UE) núm. 910/2014 relatiu a la identificació electrònica i els serveis de confiança per a les transaccions electròniques en el mercat interior (eIDAS)*. <https://eur-lex.europa.eu/eli/reg/2014/910/oj/eng>. 2014.
- [2] Wikipedia. *Health technology*. https://en.wikipedia.org/wiki/Health_technology.
- [3] Hyperledger (LF Decentralized Trust). *Hyperledger Besu Documentation*. <https://besu.hyperledger.org/>.
- [4] Nick Szabo. "Formalizing and Securing Relationships on Public Networks". A: *First Monday* 2.9 (1997). DOI: 10.5210/fm.v2i9.548. URL: <https://firstmonday.org/ojs/index.php/fm/article/view/548>.
- [5] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN: 978-0134494166.
- [6] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. ISBN: 978-0132350884.
- [7] Kent Beck et al. *Manifesto for Agile Software Development*. <https://agilemanifesto.org/>. 2001.
- [8] Gene Kim et al. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. Disponible a archive.org. IT Revolution Press, 2016. ISBN: 978-1942788003.
- [9] Jez Humble i David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010. ISBN: 978-0321601919.
- [10] GitHub, Inc. *GitHub Actions Documentation*. <https://docs.github.com/actions>.
- [11] Docker, Inc. *Docker Documentation*. <https://docs.docker.com/>.
- [12] Dokploy. *Dokploy Documentation*. <https://docs.dokploy.com/>.
- [13] Parlament Europeu i Consell de la Unió Europea. *Reglament (UE) 2016/679 general de protecció de dades (RGPD)*. <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>. 2016.
- [14] Govern d'Espanya. *Reial decret 311/2022, de 3 de maig, pel qual es regula l'Esquema Nacional de Seguretat (ENS)*. <https://www.boe.es/buscar/act.php?id=BOE-A-2022-7191>. BOE-A-2022-7191. 2022.
- [15] Validated ID. *VIDsigner — Signatura electrònica*. <https://www.validatedid.com/en/vidsigner>. 2026.
- [16] Signaturit Solutions. *Signaturit — Signatura electrònica*. <https://www.signaturit.com/>. 2026.
- [17] Adobe. *Adobe Acrobat Sign*. <https://www.adobe.com/acrobat/business/sign.html>. 2026.
- [18] Govern d'Espanya. *Llei 34/2002, d'11 de juliol, de serveis de la societat de la informació i de comerç electrònic (LSSICE)*. <https://www.boe.es/buscar/act.php?id=BOE-A-2002-13758>. BOE-A-2002-13758. 2002.

- [19] International Organization for Standardization. *ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. <https://www.iso.org/standard/27001>. 2022.
- [20] World Wide Web Consortium (W3C). *Accessible Rich Internet Applications (WAI-ARIA) 1.2*. <https://www.w3.org/TR/wai-aria-1.2/>. W3C Recommendation. 2023.
- [21] Vincent Driessen. *A successful Git branching model*. <https://nvie.com/posts/a-successful-git-branching-model/>. 2010.
- [22] National Institute of Standards and Technology. *FIPS PUB 180-4: Secure Hash Standard (SHS)*. <https://doi.org/10.6028/NIST.FIPS.180-4>. 2015.
- [23] S. Josefsson i I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA) (RFC 8032)*. IETF. <https://www.rfc-editor.org/info/rfc8032>. 2017.
- [24] NestJS. *NestJS Documentation*. <https://docs.nestjs.com/>.
- [25] TypeORM. *TypeORM Documentation*. <https://typeorm.io/>.
- [26] Vercel, Inc. *Next.js Documentation*. <https://nextjs.org/docs>.
- [27] Meta Open Source. *React*. <https://react.dev/>.
- [28] M. Jones, J. Bradley i N. Sakimura. *JSON Web Token (JWT) (RFC 7519)*. IETF. <https://www.rfc-editor.org/info/rfc7519>. 2015.
- [29] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446)*. IETF. <https://www.rfc-editor.org/info/rfc8446>. 2018.
- [30] Traefik Labs. *Traefik Proxy Documentation*. <https://doc.traefik.io/traefik/>.
- [31] PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>.
- [32] MUI. *Material UI Documentation*. <https://mui.com/material-ui/>.
- [33] Tailwind Labs. *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>.
- [34] Deuxfleurs. *Garage — An open-source distributed object storage service*. <https://garagehq.deuxfleurs.fr/>.
- [35] Amazon Web Services. *Amazon Simple Storage Service (S3) Documentation*. <https://docs.aws.amazon.com/s3/>.
- [36] Semgrep, Inc. *Semgrep Documentation*. <https://semgrep.dev/docs/>.